

Robust and Scalable Lattice-Based Distributed Key Generation for Asynchronous Networks

Linghe Yang, Jian Liu, Jingyi Cui, Guangquan Xu, Zhongshan Li, Lei Zhang, Mingzi Zuo, Kaitai Liang

Abstract—Distributed Key Generation (DKG) is essential for secure, decentralized cryptographic systems, enabling collaborative key pair generation without a trusted authority. This capability underpins critical applications such as threshold signatures and blockchain-based protocols. To achieve post-quantum security, existing robust lattice-based DKG protocols, tailored for synchronous networks, rely on complaint-based Verifiable Secret Sharing (VSS). However, these protocols lack public verifiability and compatibility with asynchronous environments, constraining their use in Byzantine fault-tolerant settings.

This paper presents LADKG, the first Lattice-Based Asynchronous Distributed Key Generation protocol designed for post-quantum secure and scalable distributed systems. LADKG integrates Asynchronous Verifiable Short Secret Sharing (AV3S) with an Approximate Asynchronous Common Subset (AACS) protocol to achieve efficient key generation. By deferring verification and leveraging deterministic approximate agreement, LADKG reduces computational and communication overhead while maintaining security and robustness. Evaluations on geo-distributed AWS EC2 clusters demonstrate that LADKG is comparable or better than classical Asynchronous Distributed Key Generation (ADKG) schemes in scalability and efficiency. Under optimistic conditions with $n = 121$ nodes, completion is achieved in 45 seconds, ensuring robust key generation for post-quantum secure applications.

Index Terms—asynchronous networks, byzantine fault tolerance, distributed key generation, lattice-based cryptography.

1. Introduction

Distributed Key Generation (DKG) protocols enable a group of mutually distrustful parties to collaboratively generate a shared cryptographic key pair without relying on a centralized authority, ensuring that no single entity possesses the complete secret key. This capability is essential for applications such as threshold signatures, secure multi-party computation (MPC), and blockchain systems, where resilience against faults and adversarial attacks is critical [1], [2]. However, most existing DKG protocols rely on pre-quantum cryptographic primitives, such as discrete logarithms and Diffie-Hellman assumptions [3]–[5], which are vulnerable to quantum computing advancements. Over the past decade, the rise of quantum computers has spurred growing interest in post-quantum cryptography [6], prompting NIST to launch its standardization process in 2016 to address quantum threats. Consequently, lattice-based DKG protocols have emerged

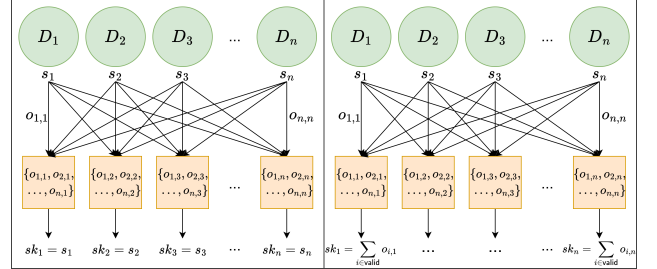


Figure 1: Key aggregation in robust lattice DKGs: (left) Additive sharing, where the secret key is the dealer’s selected secret; (right) Shamir sharing, where the secret key aggregates shares from a valid dealer set.

as a promising solution to provide post-quantum security and mitigate these vulnerabilities.

Recent advancements in lattice-based DKG and threshold signatures have explored both Identifiable Abort (IA) and Robust paradigms to enhance fault tolerance. IA mechanisms, as in [7], [8], enable efficient detection and exclusion of malicious parties with minimal overhead, allowing protocols to abort and restart upon identifying faults. In contrast, robust designs [9], [10] ensure termination and output delivery despite up to t corruptions, without requiring restarts, thereby providing stronger resilience in adversarial settings.

In distributed applications such as permissionless blockchains, however, IA paradigms for DKG are inefficient [11], [12], necessitating protocol restarts that disrupt continuous operation in Byzantine fault-tolerant (BFT) settings [13], [14]. Robust paradigms, by ensuring termination despite bounded malicious behavior, are better suited for such environments. Existing robust lattice-based DKG protocols remain in early development and rely on a complaint-based Verifiable Secret Sharing (VSS) paradigm [15]. This approach is limited to synchronous networks, as it imposes a deadline for complaints, preventing verifiers from distinguishing malicious behavior from network delays in asynchronous settings.

Classical Asynchronous DKG (ADKG) protocols commonly address asynchrony through the VSS+ACS paradigm: secrets are shared via VSS, followed by an Asynchronous Common Subset (ACS) protocol (or its equivalents)—a BFT consensus primitive—that selects dealer shares for aggregation. In lattice-based DKG, a critical challenge arises: the secret key must be short to satisfy hardness assumptions such as MSIS or MLWE. Shamir secret sharing increases key norms, rendering standard VSS unsuitable for lattice-based environments. The protocol in [9] employs Fiat-Shamir with Aborts for threshold

TABLE 1: Comparison of recent LDKG works with fault tolerance.

Protocol	Network Setting	Fault Tolerance Mechanism	Communication Complexity (per node)	Computational Complexity (per node)	Cryptographic Assumption	Setup
Ji et al. [9]	sync.	Robust	$O(\lambda n^2)$	$O(n^2)$	MLWE & MSIS & g-SPP	Authenticated channels
Esipitau et al. [10]	sync.	Robust	$O(\lambda n \log n)$	$\star O(n \log n)$	$\ddagger$ Hint-MLWE & RSIS	ROM & PKI
Pino et al. [8]	sync.	IA	$O(\lambda n)$	$O(n)$	Hint-MLWE & $\dagger$ STMSIS	ROM & PKI
This work	async.	Robust	$O(\lambda n^2 \log n)$	$\star O(n \log n)$	$\ddagger$ Hint-MLWE & RSIS & RLWE	ROM & PKI

$\ddagger$ assumes a variant, namely MatrixHint-MLWE. $\dagger$ refers to SelfTargetMSIS. $\star$ requires RS error correction in the worst case, resulting in computational complexity of $O(n^3)$.

signatures, utilizing additive sharing to aggregate keys, thereby avoiding norm amplification, with secret keys self-selected by participants. Since secret keys are self-selected, a participant that does not engage in the signing phase can have its secret key reconstructed by others to maintain the integrity of the additive sharing components. This ensures full participation from the qualified (QUAL) set is required during signing. However, the ACS component of the VSS+ACS paradigm selects a subset of nodes for aggregation, which excludes non-selected nodes—even if honest—from the signing process. As these excluded honest nodes contribute nothing to the additive sharing, their absence undermines the fairness of the protocol, rendering the VSS+ACS paradigm incompatible with this design.

In contrast, [10] employs a hash-and-sign paradigm with Shamir-based key generation, which is compatible with the VSS+ACS paradigm. Fig. 1 illustrates this distinction. To address short-norm verification, it introduces Verifiable Short Secret Sharing (V3S) for synchronous networks. The complaint-based mechanism for V3S is necessary because a malicious dealer could send valid shares only to a subset of honest nodes, leaving the remaining honest nodes without valid shares. Without such a mechanism, these nodes would be unable to ensure termination during the aggregation phase. Upon receiving an invalid share or no share at all, a node publicly reveals the symmetric encryption key shared with the dealer, thereby proving the dealer’s dishonesty to other nodes and enabling consistent exclusion of the dealer from the valid set. After the sharing phase terminates, all honest nodes possess a consistent valid set, and aggregation occurs within this set. However, asynchronous networks cannot reliably apply the condition of “receiving an invalid share or no share”. One way to address this is to use Publicly Verifiable Secret Sharing (PVSS), where the dealer proactively proves its validity without revealing private share information, thereby excluding malicious dealers. Nevertheless, unlike classical PVSS, in the lattice context, a naive solution—replacing V3S with lattice-based PVSS (e.g., [16], [17])—is ineffective, as existing lattice PVSS protocols cannot verify the short norm of the original secret. Consequently, transitioning from complaint-based V3S to a PVSS+ACS framework is non-trivial.

To achieve robust asynchronous lattice-based DKG, we explore the integration of ACS with VSS [2], [5], [18]. Recently, [19] applies this paradigm in a Distributed Key Refresh (DKR) protocol, building on a state-of-the-art ACS from [20]. Although [20] appears efficient, its performance largely relies on improvements to multi-valued

validated Byzantine agreement (MVBA), which employs a Provable Broadcast primitive and assumes the existence of a threshold signature system. However, this introduces a circular dependency, as threshold signatures presuppose a prior DKG. While DKR can leverage the threshold system from a previous round during key refresh, this ACS approach is unsuitable for a one-shot DKG. Furthermore, the MVBA in [20] depends on multiple rounds of global randomness (e.g., a randomness beacon) to circumvent the FLP impossibility theorem [21], resulting in uncertain termination. An alternative approach, approximate agreement, as proposed in [22], [23], achieves deterministic termination with a negligible probability of inconsistency without relying on randomness or threshold systems. This suggests a natural extension: an approximate ACS protocol that eliminates the need for threshold cryptosystems.

Due to the above considerations, we identify a critical limitation in existing lattice-based DKG protocols: they struggle to operate effectively in environments that more closely resemble real-world networks, namely asynchronous and Byzantine fault-tolerant settings. This prompts the following question:

Is it possible to design a robust lattice-based DKG in asynchronous networks?

Our solution: In this work, we introduce the first lattice-based asynchronous DKG (LADKG) protocol, achieving optimal resilience under Byzantine fault tolerance.

Publicly Verifiable Short Secret Sharing: As an extension of V3S, we propose Publicly Verifiable Short Secret Sharing (PV3S), leveraging a lattice-based Verifiable Encryption (VE) scheme [24] to encrypt shares, enabling their broadcast as part of a transcript with public verifiability, including short-norm compliance, completeness, and soundness. However, we avoid direct use of PV3S due to the large ciphertext size, which renders transcripts cumbersome. Drawing inspiration from a classical work of AVSS [15], we optimize this by distributing heavier content via peer-to-peer channels and broadcasting lighter verification data. Specifically, the dealer initially encrypts shares using R-LWE (a component of VE) and distributes them peer-to-peer; nodes decrypt and privately verify legitimacy, returning signatures to the dealer. Upon collecting $2t + 1$ signatures, the dealer aggregates them into a certificate and broadcasts fully VE-encrypted shares for unsigned nodes alongside this certificate, allowing verification of signature validity and unsigned share legitimacy. This approach reduces transcript communication and verification overhead. To further enhance efficiency, we adapt to real-world conditions where faults may not

always reach the tolerance limit and networks are not perpetually asynchronous. We introduce a timer during signature collection, enabling the dealer to gather as many signatures as possible while minimizing VE ciphertext generation. This optimized scheme, termed Asynchronous Verifiable Short Secret Sharing (AV3S), balances performance and robustness.

Approximate Asynchronous Common Subset: We then adopt the *Variation 1* framework from Dumbo [25], integrating it with the state-of-the-art convex agreement protocol Delphi [23] to propose an Approximate Asynchronous Common Subset (AACS). This protocol invokes a weak coin protocol once and terminates within a fixed number of rounds, delivering a consistent public subset to honest nodes with high probability. Unlike Dumbo’s *Variation 2*, which relies on threshold signatures and offers greater efficiency, we avoid it due to its dependency on prior DKG, noting that AACS’s single-shot randomness serves only for committee election, unlike the multi-round randomness required in [20] for leader selection.

LADKG: By combining AV3S with AACS, we develop the LADKG protocol, operating within the VSS+ACS paradigm while addressing the norm amplification issue absent in classical designs. Each dealer feeds its AV3S output into AACS, which produces a public subset containing shares from at least $t + 1$ honest dealers. We observe that an ACS output set of size $n - t$ is overly redundant for DKG, as perfect secrecy requires only one honest contribution to the secret key. Since the transcript of our AV3S permits deterministic subset selection, we thus rank the AACS output subset by signature count and select the first $t + 1$ entries, achieving per-node communication complexity of $O(\lambda n^2 \log n)$ and computational complexity of $O(n^2)$. Higher signature counts indicate broader receipt of legitimate shares, minimizing VE verification and decryption needs. Upon termination, each honest node obtains a threshold secret key share $s_j \in R_q^\ell$ and a public key $\mathbf{k} \in R_q$. This can be directly used in the Pelican threshold signature scheme from [10], and can also be adapted to other lattice-based threshold signatures, such as those in [9], [26], [27], by adjusting the public key computation while using the same secret key format.

Evaluation: We evaluate LADKG on geo-distributed AWS EC2 t3.medium instances across five regions, with node counts $n \in \{10, 31, 64, 121\}$. The protocol demonstrates efficient runtime, achieving optimal performance with a transcript waiting time of $\Delta = 1000$ ms, outperforming the baseline ADKG protocol [5] under optimistic network conditions. To the best of our knowledge, this marks the first experimental evaluation of a lattice-based DKG in a real-world network setting.

The rest of the paper is organized as follows: Section 2 covers the threat model and preliminaries. Section 3 presents the core building blocks. Section 4 details the LADKG protocol. Section 5 analyzes security properties. Section 6 evaluates performance. Section 7 reviews related work. Section 8 concludes with a summary and future works. Appendices B, C, and D provide security proofs and complexity analysis.

2. Models and Preliminaries

2.1. Threat Model and Network Assumptions

We adopt a framework similar to Cachin et al. [28], [29] and Abraham et al. [30], tailored for cryptographic settings in asynchronous networks. The system comprises n nodes connected via pairwise authenticated channels, with a public-key infrastructure (PKI) providing each node a unique public-secret key pair known in advance. We assume a static probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ that can corrupt up to $t < n/3$ of the $n \geq 3t + 1$ nodes, exerting full control over corrupted nodes to deviate arbitrarily from the protocol. The network is asynchronous, allowing $\mathcal{A}$ to arbitrarily delay messages, but $\mathcal{A}$ ’s computational steps between sending and delivering messages between honest nodes are bounded by a polynomial in the security parameter λ to maintain negligible error probability. This model ensures security against quantum threats, such as those exploiting Ring-LWE and MatrixHint-MLWE, by limiting the adversary’s ability to solve infeasible problems during unbounded delays.

2.2. Notations

We adopt the majority of the notation and syntax from [10]: For positive integer N , we denote $[N] = \{1, \dots, N\}$. Vectors are bold lowercase (e.g., $\mathbf{x} \in \mathbb{Z}^m$), matrices bold uppercase (e.g., $\mathbf{A} \in \mathbb{Z}^{m \times n}$), and the transpose is $\mathbf{A}^T$. The Euclidean norm is $\|\mathbf{x}\| = \sqrt{\sum_i x_i^2}$. We work over the cyclotomic ring $R = \mathbb{Z}[x]/(x^n + 1)$ for power-of-two n , and $R_q = R/qR$. We also define $S_\gamma^m = \{\mathbf{x} \in R^m \mid \|\mathbf{x}\|_\infty \leq \gamma\}$.

For $\beta \geq 2$ a power of 2, any $x \in \mathbb{Z}$ decomposes uniquely as $x = \beta x_1 + x_2$ with $x_2 \in \{-\beta/2, \dots, \beta/2 - 1\}$, $|x_1| \leq \lceil x/\beta \rceil$. For odd prime q , $\text{Decompose}_\beta : \mathbb{Z}_q \rightarrow \mathbb{Z} \times \mathbb{Z}$ takes the balanced representative $\tilde{x} \in \{-(q-1)/2, \dots, (q-1)/2\}$ and decomposes it, extended coefficient-wise to R_q . For $c \leftarrow \mathbb{Z}_q$, $(c_1, c_2) = \text{Decompose}_\beta(c)$ satisfies $|c_1| \leq \lceil (q-1)/(2\beta) \rceil$, $\mathbb{E}[c_1] = 0$, and $\mathbb{E}[c_1^2] \leq M^2 - 1/12$ for $M = 2\lceil (q-1)/(2\beta) \rceil + 1$.

We use $[\mathbf{x}]_i$ to denote the i -th Shamir share of a secret $\mathbf{x}$, and $[\mathbf{x}]$ to represent the collection of all Shamir shares. When $[\mathbf{x}]$ participates in operations, it is equivalent to performing the operation on each individual share $[\mathbf{x}]_i$ separately.

2.3. Distributions

Definition 1 (Uniform Distribution). *For a finite set S , the uniform distribution over S , denoted $U(S)$, assigns equal probability $1/|S|$ to each element in S . In particular, $U(R_q)$ denotes the uniform distribution over the ring R_q , where each coefficient is sampled uniformly from $\mathbb{Z}_q$.*

Definition 2 (Discrete Gaussians). *Let $\Lambda \subseteq R^n$ be a full-rank lattice in R^n . For any $\sigma > 0$ and $\mathbf{c} \in R^n$, the discrete Gaussian distribution over Λ with parameter σ and center $\mathbf{c}$, denoted $D_{\Lambda, \sigma, \mathbf{c}}$, is defined as follows: for any $\mathbf{y} \in \Lambda$,*

$$D_{\Lambda, \sigma, \mathbf{c}}(\mathbf{y}) = \frac{\rho_{\sigma, \mathbf{c}}(\mathbf{y})}{\rho_{\sigma, \mathbf{c}}(\Lambda)}, \quad (1)$$

where $\rho_{\sigma, \mathbf{c}}(\mathbf{x}) = \left(\frac{1}{\sqrt{2\pi}\sigma}\right)^n \exp\left(-\frac{\|\mathbf{x}-\mathbf{c}\|^2}{2\sigma^2}\right)$ is the Gaussian function, and $\rho_{\sigma, \mathbf{c}}(\Lambda) = \sum_{\mathbf{y} \in \Lambda} \rho_{\sigma, \mathbf{c}}(\mathbf{y})$ is the normalization factor. When $\mathbf{c} = \mathbf{0}$, we denote it by D_σ^n .

2.4. Hardness Assumptions

Definition 3 (Ring-LWE). Let n be a power of 2, q an odd prime, χ a distribution over R . The advantage $\text{Adv}_{\text{Ring-LWE}_A}(\lambda)$ of an adversary $\mathcal{A}$ in the Ring Learning with Errors problem, denoted $\text{Ring-LWE}_{n,q,\chi}$, is defined as:

$$\left| \Pr[1 \leftarrow \mathcal{A}((a_i, a_i \cdot s + e_i)_{i \in [m]})] - \Pr[1 \leftarrow \mathcal{A}((a_i, u_i)_{i \in [m]})] \right|, \quad (2)$$

where $a_i, u_i \leftarrow U(R_q)$, $s \leftarrow \chi$, $e_i \leftarrow \chi$, and $m = \text{poly}(\lambda)$. The $\text{Ring-LWE}_{n,q,\chi}$ assumption suggests that no efficient adversary $\mathcal{A}$ can gain a significant edge in this guessing game.

Definition 4 (MatrixHint-MLWE). Let k, ℓ, q , and Q be integers. Imagine $\mathcal{P}_{sk}$ as a way to pick random vectors from $\mathbb{Z}^{k+\ell}$, and $\{\mathcal{P}_i^{(p)}\}_{i \in [Q]}$ as a set of methods to generate random noise vectors, one for each i from 1 to Q . Also, consider $\mathcal{M}$ as a process that creates a collection of challenge matrices from $(\mathbb{Z}^{np(k+\ell) \times np(k+\ell)})^Q$. We define the advantage $\text{Adv}_{\text{MatrixHint-MLWE}_A}(\lambda)$ of an adversary $\mathcal{A}$ in the Matrix Hint Module Learning with Errors problem, denoted $\text{MatrixHint-MLWE}_{k,\ell,q,Q,\mathcal{P}_{sk},\{\mathcal{P}_i^{(p)}\}_{i \in [Q]},\mathcal{M}}$, as follows:

$$\left| \Pr[1 \leftarrow \mathcal{A}(\mathbf{A}, [\mathbf{I}_k \ \mathbf{A}] \cdot \mathbf{s}, (\mathbf{M}_i, \mathbf{z}_i)_{i \in [Q]})] - \Pr[1 \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{u}, (\mathbf{M}_i, \mathbf{z}_i)_{i \in [Q]})] \right|. \quad (3)$$

As in [10]: The $\text{MatrixHint-MLWE}_{k,\ell,q,Q,\mathcal{P}_{sk},\{\mathcal{P}_i^{(p)}\}_{i \in [Q]},\mathcal{M}}$ assumption suggests that no efficient adversary $\mathcal{A}$ can gain a significant edge in this guessing game.

2.5. Primitive Definitions

We review key primitives utilized in our protocol to ensure clarity and consistency across subsequent sections.

Definition 5 (Reliable Broadcast [31]). A protocol for nodes $1, \dots, n$ where each node can invoke a broadcaster to disseminate a message m is a Reliable Broadcast (RBC) protocol if it satisfies the following properties:

- **Agreement:** For any two honest nodes p_1 and p_2 , if p_1 outputs message m_1 and p_2 outputs message m_2 , then $m_1 = m_2$.
- **Validity:** If an honest node p invokes RBC to broadcast message m , all other honest nodes eventually output m .
- **Totality:** If an honest node p outputs message m , all other honest nodes eventually output m .

Definition 6 (Weak Coin). Following a definition akin to [3], a protocol for nodes $1, \dots, n$ where each node can invoke a coin-tossing protocol to output a value $x \in \mathcal{D}$ is a Weak Coin (WC) protocol if it satisfies:

- **α -Uniformity:** Whenever the protocol is invoked, for any two honest nodes p_1 and p_2 outputting r_1

and r_2 respectively, the probability that $r_1 = r_2$ exceeds α .

- **Termination:** All honest nodes participating in the protocol terminate and output a value.

Definition 7 (Reed-Solomon Code [32]). A Reed-Solomon (RS) code over a finite field $\mathbb{F}_q$ is an error-correcting code defined by parameters (n, k) , where n is the code length (with $n \leq q - 1$), k is the number of information symbols, and the minimum distance $\delta = n - k + 1$. It operates on symbols from $\mathbb{F}_q$ and can detect up to $\delta - 1$ errors, correct up to $\lfloor (\delta - 1)/2 \rfloor$ errors, or correct up to $\delta - 1$ erasures at known positions.

Definition 8 (Lattice-based Verifiable Encryption). A lattice-based Verifiable Encryption (VE) scheme comprises the following components:

- $(\text{pk}, \text{sk}) \leftarrow \text{Setup}(1^\lambda)$: A probabilistic function that generates public key pk and secret key sk for security parameter λ .
- $(\text{ct}, \text{st}) \leftarrow \text{RLWE-Enc}(\text{pk}, \mathbf{m})$: A probabilistic function that encrypts message $\mathbf{m}$ using pk , producing ciphertext ct and state st .
- $\pi \leftarrow \text{VE-Prove}(\text{pk}, \mathbf{B}, \mathbf{u}, \text{st})$: A probabilistic function that generates a zero-knowledge proof π using Fiat-Shamir with aborts.
- $0 \text{ or } 1 := \text{VE-Verify}(\text{pk}, \mathbf{B}, \mathbf{u}, \text{ct}, \pi)$: A deterministic function that verifies the ciphertext and proof, returning 1 (accept) or 0 (reject).
- $\mathbf{m} := \text{RLWE-Dec}(\text{sk}, \text{ct})$: A deterministic function that decrypts ciphertext ct using sk , yielding message $\mathbf{m}$.
- $\mathbf{m} \text{ or } \perp := \text{VE-Dec}(\text{pk}, \text{sk}, \text{ct}, \pi)$: A deterministic function that verifies the proof and, if valid, decrypts to return $\mathbf{m}$, else outputs $\perp$.

We adopt a lattice-based VE scheme based on [24], enabling verification that a ciphertext ct encrypts a witness $\mathbf{m} \in S_\gamma^k$ satisfying $\mathbf{B}\mathbf{m} = \mathbf{u} \pmod{q}$, where $\mathbf{B} \in R_q^{l \times k}$ and $\mathbf{u} \in R_q^l$, without decryption. For our protocol, where not all shares require public verifiability, we utilize the underlying Ring-LWE encryption and decryption components. The original scheme relaxes decryption to yield a witness $(\tilde{\mathbf{m}}, \tilde{c})$ with small coefficients satisfying $\mathbf{B}\tilde{\mathbf{m}} = \tilde{c}\mathbf{u} \pmod{q}$, ensuring $\tilde{\mathbf{m}}/\tilde{c} = \mathbf{m}'/c' \pmod{q}$ for valid witnesses. With $q \equiv 5 \pmod{8}$ and $\tilde{c}$ having bounded norm (Hamming weight at most 36 per [24]), $\tilde{c}$ is invertible in R_q , allowing recovery of the original $\mathbf{m} = \tilde{\mathbf{m}} \cdot \tilde{c}^{-1} \pmod{q}$ in VE-Dec . This adjustment supports our application, where honest behavior is incentivized, and adversary queries to the random oracle are bounded.

3. Building Blocks

We introduce the core techniques that underpin our DKG for asynchronous distributed systems. These building blocks include a publicly verifiable short secret sharing, an asynchronous verifiable short secret sharing, and an approximate asynchronous common subset protocol.

3.1. Public Verifiable Short Secret Sharing

The Verifiable Short Secret Sharing (V3S) scheme, introduced in [10], provides a lattice-based mechanism

for sharing short secrets in synchronous networks. It leverages the MatrixHint-MLWE hardness assumption and the Johnson-Lindenstrauss lemma [33] to approximately prove the short-norm property of a secret vector: given the relation $\mathbf{u} = \mathbf{R}\mathbf{x} + \mathbf{y} \pmod{q}$, verifying that $\|\mathbf{u}\|$ is short implies that $\|\mathbf{x}\|$ is likely short as well. However, the original V3S lacks public verifiability, as shares are transmitted via reliable channels without enabling global validation. To address this, we propose PV3S, a publicly verifiable variant, by integrating the lattice-based VE scheme. This allows all participants to verify the legitimacy of all encrypted shares in a broadcast transcript without decryption, akin to classical PVSS. Our construction maintains the approximate shortness proof while ensuring compatibility with public verifiability.

Algorithm 1 details the PV3S procedures. In the Share phase, the dealer generates Shamir shares for the secret $\mathbf{x}$ and a Gaussian mask $\mathbf{y}$, encrypts decomposed low-norm representations using VE, constructs a Merkle tree over the ciphertexts for verifiable randomness, derives the submersion matrix $\mathbf{R}$, and computes the relation proofs. The Verify phase enables public validation of the transcript, including norm checks (bounded by B') and VE verifications, with private decryption for one's own share. Reconstruction uses Lagrange interpolation, while robust reconstruction employs Reed-Solomon error correction.

To enable verifiable encryption of Shamir shares in PV3S, which inherently amplify the norm of the original short secrets $\mathbf{x}$ and $\mathbf{y}$ due to polynomial interpolation, we employ a gadget decomposition technique common in lattice-based cryptography. For each share $[\mathbf{x}]_i$ and $[\mathbf{y}]_i$, we decompose them using gadget matrices $\mathbf{G}_x$ and $\mathbf{G}_y$ (defined as $\mathbf{G} = \mathbf{I} \otimes g^T$, where $0 < b \leq \gamma + 1$, $\delta = \lceil \log_b q \rceil$, and $g^T = (1, b, b^2, \dots, b^\delta)$), yielding short vectors $\mathbf{s}_{x,i}$ and $\mathbf{s}_{y,i}$ such that $[\mathbf{x}]_i = \mathbf{G}_x \mathbf{s}_{x,i}$ and $[\mathbf{y}]_i = \mathbf{G}_y \mathbf{s}_{y,i}$. These short vectors are concatenated into $\mathbf{m}_i = [\mathbf{s}_{x,i}; \mathbf{s}_{y,i}]$ (Line 3), satisfying the low-norm requirement $\mathbf{m}_i \in S_\gamma^k$ for VE encryption. The relation matrix is adjusted to $\mathbf{B} = [\mathbf{R}\mathbf{G}_x \mid \mathbf{G}_y]$, ensuring

$$\mathbf{u}_i = \mathbf{B}\mathbf{m}_i \pmod{q}, \quad (4)$$

preserves the original submersion equation

$$\mathbf{u}_i = \mathbf{R}[\mathbf{x}]_i + [\mathbf{y}]_i \pmod{q}. \quad (5)$$

(Line 6) During verification and reconstruction, the recipient decrypts $\mathbf{m}_j$ and recovers $[\mathbf{x}]_j = \mathbf{G}_x \mathbf{s}_{x,j}$, maintaining linearity and security under lattice assumptions (Line 24).

The PV3S scheme must satisfy several key properties to ensure its reliability and security in an asynchronous distributed setting. These properties are outlined as:

- **Correctness:** If the dealer L is honest with a secret s , the sharing phase ensures all honest nodes eventually receive a share of s . Upon completion, if all honest nodes initiate the reconstruction phase, they unanimously output s .
- **Completeness:** If any honest node completes the sharing phase, there exists a secret $\tilde{s} \in \mathbb{F}$ such that all honest nodes eventually obtain a share of $\tilde{s}$. Furthermore, $\tilde{s}$ is guaranteed to be reconstructed during the reconstruction phase.
- **Secrecy:** If the dealer L is honest, the adversary's view during the sharing phase in the real proto-

Algorithm 1 Publicly Verifiable Short Secret Sharing (PV3S)

Input: λ , VE public keys $\{pk_i\}_{i=1}^N$, VE secret key sk_i for each party i

PV3S.Share($N, T, \mathbf{x}$):

- 1: Sample $\mathbf{y} \sim D_{\sigma_y}^d$
 - 2: Generate T -out-of- N Shamir shares: $[\mathbf{x}] \parallel [\mathbf{y}]$
 - 3: Decompose $[\mathbf{x}]_i \parallel [\mathbf{y}]_i$ into $\mathbf{s}_{x,i} \parallel \mathbf{s}_{y,i}$, set $\mathbf{m}_i \leftarrow [\mathbf{s}_{x,i}; \mathbf{s}_{y,i}]$, $(ct_i, st_i) \leftarrow \text{RLWE-Enc}(pk_i, \mathbf{m}_i)$ for $i \in [N]$
 - 4: Build Merkle tree over $\{ct_i\}_{i=1}^N$ with root h and proofs $\{\text{mpf}_i\}_{i=1}^N$
 - 5: Derive $\mathbf{R} \leftarrow H_R(h)$ and build $\mathbf{B} \leftarrow [\mathbf{R}\mathbf{G}_x \mid \mathbf{G}_y]$
 - 6: Compute $\mathbf{u}_i \leftarrow \mathbf{B}\mathbf{m}_i \pmod{q}$, $\pi_i \leftarrow \text{VE-Prove}(pk_i, \mathbf{B}, \mathbf{u}_i, st_i)$ for $i \in [N]$
 - 7: Set transcript $\mathcal{T} \leftarrow \{h, \{ct_i, \mathbf{u}_i, \pi_i, \text{mpf}_i\}_{i=1}^N\}$
 - 8: **return** $\mathcal{T}$
-

PV3S.Verify($\mathcal{T}$):

- 9: Parse $h, \{ct_i, \mathbf{u}_i, \pi_i, \text{mpf}_i\}_{i=1}^N \leftarrow \mathcal{T}$
 - 10: Reconstruct $\mathbf{u} \leftarrow \text{PV3S.Reconstruct}(\{\mathbf{u}_i\}_{i=1}^N)$
 - 11: **if** $\mathbf{u} = \perp$ **then return** $\perp$
 - 12: **end if**
 - 13: Derive $\mathbf{R} \leftarrow H_R(h)$
 - 14: **if** (mpf_i is not in h for any $i \in [N]$) **then**
 - 15: **return** $\perp$
 - 16: **end if**
 - 17: Build $\mathbf{B} \leftarrow [\mathbf{R}\mathbf{G}_x \mid \mathbf{G}_y]$
 - 18: **if** ($\text{VE-Verify}(pk_i, \mathbf{B}, \mathbf{u}_i, ct_i, \pi_i)$ is 0 for any $i \in [N]$) **or** ($\|\mathbf{u}\| > B'$) **then**
 - 19: **return** $\perp$
 - 20: **end if**
 - 21: For own j : $\mathbf{m}_j \leftarrow \text{VE-Dec}(pk_j, sk_j, ct_j, \pi_j)$
 - 22: **if** $\mathbf{m}_j = \perp$ **then return** $\perp$
 - 23: **end if**
 - 24: Recover $[\mathbf{x}]_j \leftarrow \mathbf{G}_x \mathbf{s}_{x,j}$ from $\mathbf{m}_j = [\mathbf{s}_{x,j}; \mathbf{s}_{y,j}]$
 - 25: **return** $[\mathbf{x}]_j$
-

PV3S.Reconstruct($[\mathbf{x}]_I$), with $|I| \geq T$:

- 26: $[\mathbf{x}]_J \leftarrow$ any T -sized subset of $[\mathbf{x}]_I$
 - 27: Lagrange interpolate over $[\mathbf{x}]_J$ to get the unique polynomial $P(\cdot)$
 - 28: **if** $P(k) \neq [\mathbf{x}]_k$ for any $[\mathbf{x}]_k \in [\mathbf{x}]_I / [\mathbf{x}]_J$ **then return** $\perp$
 - 29: **end if**
 - 30: **return** $P(0)$
-

PV3S.RobustReconstruct($[\mathbf{x}]_I$), with $|I| \geq T$:

- 31: Interpret $[\mathbf{x}]_I$ as a Reed-Solomon codeword of $\begin{cases} \text{block length } T \\ \text{message length } |I| \end{cases}$
 - 32: Run error correction on $[\mathbf{x}]_I$
 - 33: **return** $\text{PV3S.Reconstruct}([\mathbf{x}]_I)$
-

col can be simulated using shares from up to t corrupted parties.

- **Termination:** All honest nodes are guaranteed to eventually complete the sharing phase.

3.2. Asynchronous Verifiable Short Secret Sharing (AV3S)

The scheme in [15] presents an optimized PVSS for $n = 3t + 1$ parties with at most t malicious ones. It employs a Shamir scheme with a $2t$ -degree polynomial for $t + 1$ -threshold sharing. Verifiers validate their shares and return signatures to the dealer, who collects $n - t$

signatures to form a certificate and publicly reveals the remaining t shares in plaintext to ensure completeness. This design motivates an implicit complaint mechanism, where the dealer interprets unsigned shares as complaints, necessitating revelation of correct shares for verification. While secrecy holds (requiring $2t + 1$ points for reconstruction), we argue that this approach fails in DKG settings under the VSS-ACS paradigm. Specifically, ACS selects a subset of dealers for aggregation, but in an asynchronous network, dealers in this subset may collectively fail to receive signatures from a particular honest party j . Consequently, all such dealers would publicly reveal party j 's shares in plaintext, exposing j 's threshold secret key across the network and compromising security. To address this, drawing from the dual-threshold variant in [15] that uses VE, we propose AV3S, adapting PV3S for asynchronous networks by deferring VE proofs to at most t shares, reducing computational overhead to roughly one-third. An optional Δ timer further minimizes proofs by collecting additional signatures, assuming bounded delays and sub-maximal faults; optimistically, no supplementary shares are needed if all honest parties sign timely.

As algorithm 2 details, AV3S modifies PV3S as follows: Share encrypts shares without proofs, returning encrypted shares and public data $(h, \mathbf{u}_i)$. Verify decrypts and checks own share in plaintext, returning a signature on h if valid. Confirm collects $n - t$ signatures, waits Δ for more, then generates proofs only for unsigned shares in supplementary data. Unlike PV3S, the transcript in AV3S comprises these signatures and supplementary shares. TranscriptVerify publicly checks completeness ($P = [N] \setminus S$), signatures, and VE proofs for P . Reconstruction mirrors PV3S.

3.3. Approximate Asynchronous Common Subset (AACS)

To achieve efficient consensus in asynchronous setting, we build upon the Asynchronous Common Subset (ACS) protocol from Dumbo [25], but introduce approximations to reduce communicational overhead while maintaining practical efficiency. The resulting protocol, which we call Approximate Asynchronous Common Subset (AACS), relaxes the strict guarantees of ACS with negligible inconsistency probabilities, making it suitable for distributed systems where perfect agreement is not strictly necessary but high probability correctness is sufficient.

In the standard ACS protocol of Dumbo1 [25], each node broadcasts its input via a Reliable Broadcast (RBC) instance. Then, a set of κ leaders (selected via a global coin protocol) each propose an index set of at least $n - f$ delivered RBC outputs through another round of indexed RBCs. Finally, κ Asynchronous Binary Agreement (ABA) instances are run to decide which leader's proposal to adopt, ensuring a common subset with high probability. This design reduces the number of ABA instances from n (as in HoneyBadgerBFT [34]) to κ . However, both Dumbo [25] and its optimized variant Speeding Dumbo [20] rely on threshold signatures. To avoid this, we propose AACS, eschewing threshold cryptography.

Our AACS refines this structure to achieve determinism and lower communication costs, see Fig. 2:

Algorithm 2 Asynchronous Verifiable Short Secret Sharing (AV3S)

Input: λ , VE public keys $\{pk_i\}_{i=1}^N$, VE secret key sk_i for each party i ; Sign public keys $\{sig-pk_i\}_{i=1}^N$, Sign secret key $sig-sk_i$ for each party i

AV3S.Share($N, T, \mathbf{x}$):

Same as PV3S.Share until line 4 (including sampling $\mathbf{y}$, Shamir shares, decomposition, and RLWE-Enc for all i)

- 1: Compute $\mathbf{u}_i = \mathbf{R}[\mathbf{x}]_i + [\mathbf{y}]_i$ for $i \in [N]$
 - 2: **return** $\{\mathbf{ct}_i, \mathbf{u}_i, \text{mpf}_i, h\}_{i=1}^N$
-

AV3S.Verify($\{\mathbf{ct}_j, \mathbf{u}_j, \text{mpf}_j, h\}$):

Same as PV3S.Verify from line 10 to 16 (including reconstructing $\mathbf{u}$, Merkle verification)

- 3: $\mathbf{m}_j \leftarrow \text{RLWE-Dec}(sk_j, \mathbf{ct}_j)$
 - 4: Recover $[\mathbf{x}]_j, [\mathbf{y}]_j$ from $\mathbf{m}_j = [s_{x,j}; s_{y,j}]$
 - 5: **if** $(\mathbf{u}_j \neq \mathbf{R}[\mathbf{x}]_j + [\mathbf{y}]_j) \vee (\|\mathbf{u}\| > B')$ **then return** $\perp$
 - 6: **end if**
 - 7: $\text{sig}_j \leftarrow \text{Sign}(sig-sk_j, h)$
 - 8: **return** $\text{sig}_j, [\mathbf{x}]_j$
-

AV3S.Confirm(collected signatures):

- 9: Wait to collect at least $n - t$ valid signatures $\{\text{sig}_i\}_{i \in S}$
 - 10: Start timer Δ and continue collecting additional signatures
 - 11: Upon timer expiry, let S be the set of indices with valid signatures
 - 12: Let $P = [N] \setminus S$ $\triangleright$ Remaining indices, $|P| \leq t$
 - 13: Form certificate $\text{cert} \leftarrow \{\text{sig}_i\}_{i \in S}$
 - 14: **for** $i \in P$ **do**
 - 15: $\pi_i \leftarrow \text{VE-Prove}(pk_i, \mathbf{B}, \mathbf{u}_i, \text{st}_i)$
 - 16: Set $\text{sup_share}_i \leftarrow \{\mathbf{ct}_i, \mathbf{u}_i, \pi_i, h, \text{mpf}_i\}$
 - 17: **end for**
 - 18: Set transcript $\mathcal{T} \leftarrow \{\text{cert}, \{\text{sup_share}_i\}_{i \in P}\}$
 - 19: **return** $\mathcal{T}$
-

AV3S.TranscriptVerify($\mathcal{T}$):

- 20: Parse $\{\text{sig}_i\}_{i \in S}, \{\text{sup_share}_i\}_{i \in P} \leftarrow \mathcal{T}$
 - 21: Derive $\mathbf{R} \leftarrow H_R(h)$ and build $\mathbf{B} \leftarrow [\mathbf{R}\mathbf{G}_x \mid \mathbf{G}_y]$
 - 22: **if** $P \neq [N] \setminus S \vee |S| < n - t$ **then return** 0
 - 23: **end if**
 - 24: **if** $\text{VerifySig}(sig-pk_i, h, \text{sig}_i) = 0$ for any $i \in S$ **then**
 - 25: **return** 0
 - 26: **end if**
 - 27: **for** $i \in P$ **do**
 - 28: Parse $\mathbf{ct}_i, \mathbf{u}_i, \pi_i, \text{mpf}_i \leftarrow \text{sup_share}_i$
 - 29: **if** mpf_i not in $h \vee \text{VE-Verify}(pk_i, \mathbf{B}, \mathbf{u}_i, \mathbf{ct}_i, \pi_i) = 0$ **then return** 0
 - 30: **end if**
 - 31: **end for**
 - 32: **return** 1
-

AV3S.Reconstruct($[\mathbf{x}]_I$), with $|I| \geq T$:

Same as PV3S.Reconstruct

AV3S.RobustReconstruct($[\mathbf{x}]_I$), with $|I| \geq T$:

Same as PV3S.RobustReconstruct

- **Leader Selection via Weak Coin:** Instead of a perfect global coin, we employ a weak coin protocol [35] for leader selection, offering the advantage of lighter cryptographic primitives and reduced communication complexity and rounds.
- **Content and Index RBC Rounds:** These remain unchanged from Dumbo1. Each node broadcasts its input in the first RBC round. Leaders then broadcast indices of at least $n - f$ delivered inputs

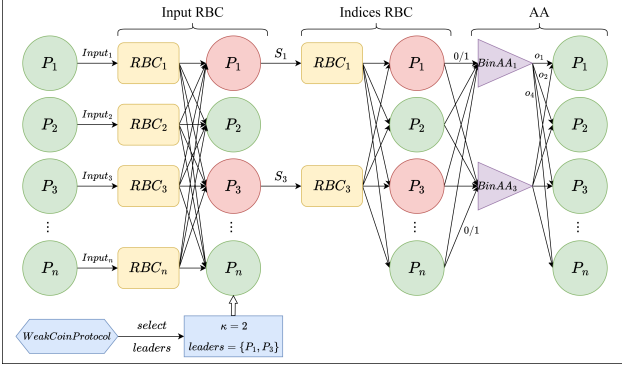


Figure 2: Similar to Dumbo1, AACS consists of three stages: Input RBC, indices RBC, and AA. The indices RBC is only performed by leaders elected via the weak coin protocol, after which parties only participate in the BinAA instances of these leaders. The figure shows an example with $\kappa = 2$, where party 1 and party 3 are selected as leaders.

in the second RBC round.

- **Approximate ABA:** We replace each ABA instance with an instance of BinAA in the Delphi protocol [23], which is the state-of-the-art deterministic asynchronous approximate agreement protocol for convex validity. In BinAA, nodes input values from $\{0, 1\}$ and output $o_i \in [0, 1]$ that satisfy $|o_i - o_j| < \epsilon$ for a small length of ϵ . To simulate binary agreement (deciding 0 or 1), nodes input either 0 or 1 based on their view of the leader's proposal. Let $MID = 0.5$. Upon outputting o_i from BinAA, a node interprets $o_i \geq MID$ as 1 and $o_i < MID$ as 0. This leverages BinAA's approximate agreement to mimic binary decisions, as analyzed in [23], where clustered inputs around 0 or 1 yield outputs tightly grouped below or above MID.

The AACS protocol satisfies the following properties:

- **Approximate Agreement:** If an honest node outputs a set V , then any other honest node outputs a set V' such that the probability that $V \neq V'$ (i.e., V and V' differ in at least one element) is at most $\text{negl}(\lambda)$.
- **Validity:** If an honest node outputs a set V , then $|V| > n - f$ and V contains the inputs of at least $n - 2f$ honest nodes.
- **Totality:** If $n - f$ honest nodes have an input, then all honest nodes produce an output.

The approximation in BinAA may introduce inconsistency in binary decisions. The overall inconsistency probability is bounded by $P_{wc} + \kappa \cdot P_{\text{single}}$, where P_{wc} is the weak coin failure probability, and P_{single} is the per-BinAA inconsistency probability. The detailed analysis is presented in Section 5.2.

This design inherits Dumbo1's $O(n^2)$ communication complexity per instance while eliminating randomization in agreement, making AACS particularly efficient for distributed systems with light cryptographic primitives.

4. Protocol Design

We present our main protocol, the Lattice-Based Asynchronous Distributed Key Generation (LADKG), which enables a set of n nodes to collaboratively generate a threshold key pair in a fully asynchronous network while tolerating up to $t < n/3$ Byzantine faults. LADKG leverages the building blocks introduced in Section 3, combining the efficiency of AV3S for secret sharing with the deterministic consensus provided by AACS. The overall structure of LADKG is illustrated in Fig. 3.

The detailed protocol is outlined in Algorithm 3. Below, we describe each phase of the protocol, highlighting its purpose and key mechanisms.

Share Phase, each node i acts as a dealer and samples a secret vector s_i from the discrete Gaussian distribution D_{σ}^d . It then invokes AV3S.Share to generate encrypted shares and distributes them to all nodes via reliable channels (Line 3). This phase ensures that secrets are shared without immediate public proofs, deferring costly verifications to later stages.

Reply Phase occurs when a node j receives a share from dealer i . Node j verifies its own share using AV3S.Verify , which involves private decryption and local checks (Line 4). If valid, it signs the Merkle root h_i and sends the signature back to the dealer, while storing the share in its local set $\mathcal{X}_j$. This phase collects acknowledgments efficiently without global broadcasts.

Certify Phase, each dealer i runs AV3S.Confirm in the background to gather at least $n - t$ signatures. Once a transcript $\mathcal{T}_i$ is formed (potentially including supplementary VE proofs for up to t shares), it is input to the AACS protocol (Line 11). AACS ensures that a common subset of at least $n - t$ transcripts is agreed upon deterministically, providing totality and relaxed validity.

Aggregate Phase is executed by each node j upon receiving the common subset $\{\mathcal{T}_k\}_{k \in S}$ from AACS. Inspired by the approach in [19], we defer the verification of VSS transcripts (via $\text{AV3S.TranscriptVerify}$, Line 16) until after AACS outputs the common subset. This avoids unnecessary computational waste on transcripts that may not be selected by the consensus, as only those in the agreed subset need validation.

While the common subset size exceeds $n - t$, to ensure privacy, only $t + 1$ transcripts need to be aggregated for the threshold key. We observe that in scenarios where the actual number of malicious nodes is less than t , some transcripts may contain more signatures (in $\mathcal{C}_t$) and fewer supplementary shares (in $\{\text{sup_share}_i\}$). Since verifying signatures is computationally faster than verifying VE proofs in supplementary shares, we employ a deterministic sorting algorithm (Line 13) to reorder the common subset. We then sequentially verify and select the first $t + 1$ valid transcripts, forming $\mathcal{DT}_j$. This minimizes overall verification costs by prioritizing transcripts with fewer proofs.

Next, for any missing shares (indices in $\langle \mathcal{DT}_j \rangle \setminus \langle \mathcal{X}_j \rangle$), node j decrypts the supplementary data in the transcripts to recover them (Line 27). A salt is obtained from the weak coin protocol (which can be shared with AACS to reuse randomness). The public ring element $a = H_a(\text{salt})$ is computed, and the threshold secret key share is aggregated as $s_j = \sum_{k \in \langle \mathcal{DT}_j \rangle} \llbracket s_k \rrbracket_j$. The public key share is

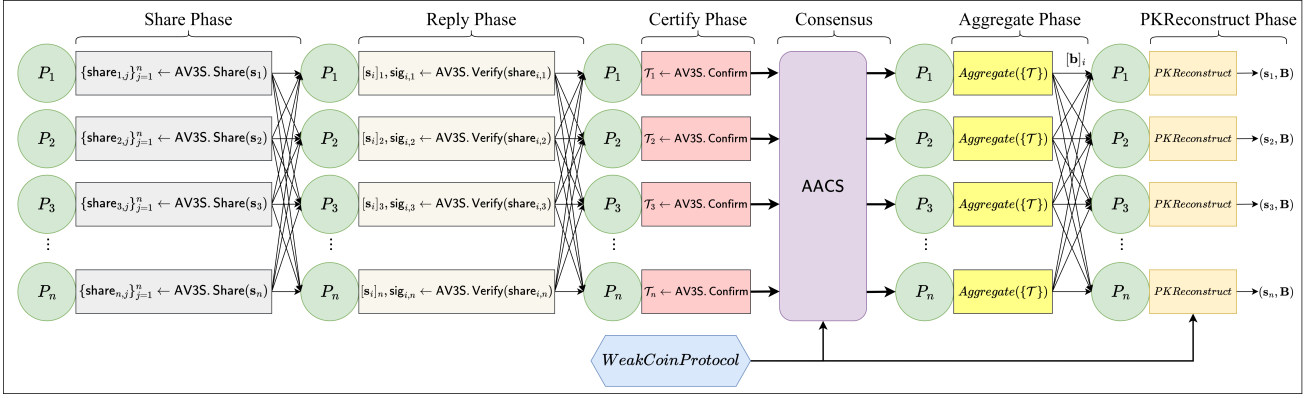


Figure 3: The overall structure of the Lattice-Based Asynchronous Distributed Key Generation (LADKG) protocol, showing the flow from secret sharing to threshold key reconstruction.

Algorithm 3 Lattice-Based Asynchronous Distributed Key Generation (LADKG)

Input: Security parameter λ , n nodes with $t < n/3$, VE keys $\{pk_i, sk_i\}_{i=1}^n$, Sign keys $\{sig-pk_i, sig-sk_i\}_{i=1}^n$

Share Phase (executed by each node i as dealer):

- 1: Sample $s_i \sim D_\sigma^l$ $\triangleright$ Short secret from discrete Gaussian
- 2: $\{share_{i,j}\}_{j=1}^n \leftarrow \text{AV3S.Share}(n, t+1, s_i)$
- 3: Send $share_{i,j}$ to node j via reliable channel for $j \in [n]$

Reply Phase (executed by each node j upon receiving from dealer i):

- 4: $[s_i]_j, sig_{i,j} \leftarrow \text{AV3S.Verify}(share_{i,j})$
- 5: **if** $[s_i]_j \neq \perp$ **then**
- 6: Send $sig_{i,j}$ to dealer i
- 7: Store $[s_i]_j$ in $\mathcal{X}_j$
- 8: **end if**

Certify Phase (executed by each dealer i):

- 9: Run AV3S.Confirm in background to collect signatures on h_i
- 10: Upon $\mathcal{T}_i \leftarrow \text{AV3S.Confirm}$ output
- 11: Input $\mathcal{T}_i$ to $\text{AACS}(\mathcal{T}_i)$

Aggregate Phase (executed by each node j):

- 12: Wait for AACS to output a common subset $\{\mathcal{T}_k\}_{k \in S}$ with $|S| \geq n-t$
- 13: Sort $\{\mathcal{T}_k\}_{k \in S}$: first by decreasing $|\text{cert}_k|$ (cert length), then by increasing dealer ID k

- 14: Initialize $\mathcal{D}\mathcal{T}_j \leftarrow \emptyset$
- 15: **for** each $\mathcal{T}_k$ in sorted order **do**
- 16: **if** $\text{AV3S.TranscriptVerify}(\mathcal{T}_k) = 1$ **then**
- 17: Add $\mathcal{T}_k$ to $\mathcal{D}\mathcal{T}_j$
- 18: **end if**
- 19: **if** $|\mathcal{D}\mathcal{T}_j| = t+1$ **then break**
- 20: **end if**
- 21: **end for**
- 22: Let $\langle \mathcal{D}\mathcal{T}_j \rangle \leftarrow \{k \mid \mathcal{T}_k \in \mathcal{D}\mathcal{T}_j\}$ $\triangleright$ Indices of valid transcripts
- 23: Let $\langle \mathcal{X}_j \rangle \leftarrow \{k \mid [s_k]_j \in \mathcal{X}_j\}$ $\triangleright$ Indices of received shares
- 24: **for** $k \in \langle \mathcal{D}\mathcal{T}_j \rangle \setminus \langle \mathcal{X}_j \rangle$ **do**
- 25: Parse sup_share from $\mathcal{T}_k$
- 26: $\mathbf{m}_{k,j} \leftarrow \text{VE-Dec}(pk_j, sk_j, \mathbf{v}_{k,j}, \mathbf{w}_{k,j}, c_{k,j}, \mathbf{z}_{k,j})$ from sup_share (if needed)
- 27: Recover $[s_k]_j$ from $\mathbf{m}_{k,j}$
- 28: Add $[s_k]_j$ to $\mathcal{X}_j$
- 29: **end for**
- 30: Obtain salt from WC protocol, compute $a \leftarrow H_a(\text{salt})$
- 31: Compute threshold secret key share $s_j \leftarrow \sum_{k \in \langle \mathcal{D}\mathcal{T}_j \rangle} [s_k]_j$
- 32: Compute threshold public key share $\mathbf{b}_j \leftarrow \beta - [1 \ a] \cdot s_j$
- 33: Broadcast $\mathbf{b}_j$

PKReconstruct Phase (executed by each node j):

- 34: Collect $\{\mathbf{b}_k\}_{k \in Q}$ with $|Q| \geq 2t+1$
 - 35: $\mathbf{k} \leftarrow \text{AV3S.RobustReconstruct}(\{\mathbf{b}_k\}_{k \in Q})$
 - 36: **return** $(s_j, \mathbf{k})$ $\triangleright$ Threshold key pair
-

then $\mathbf{b}_j = \beta - [1 \ a] \cdot s_j$, where β ensures the shortness of elements in signature verification, as required in [36]. Finally, $\mathbf{b}_j$ is broadcast.

PKReconstruct Phase, each node collects at least $2t+1$ public key shares and uses $\text{AV3S.RobustReconstruct}$ to recover the threshold public key $\mathbf{k}$, paired with its private share s_j . This phase leverages Reed-Solomon error correction for robustness against faulty $\mathbf{b}_j$.

5. Analysis

5.1. Security of Verifiable Short Secret Sharings

Lemma 1. *PV3S satisfies correctness, completeness, and secrecy under MatrixHint-MLWE.*

Proof. An honest dealer generates a valid transcript with all shares of s , enabling recipients to obtain shares. Reconstruction with $\geq 2t+1$ shares exceeds the $t+1$ threshold, using Reed-Solomon to correct up to $(|I| - T)/2$ errors, handling t faults for unanimous s .

The Merkle tree and submersion matrix $\mathbf{R}$ ensure consistent relations $\mathbf{u}_i = \mathbf{B}\mathbf{m}_i$, publicly verifiable. Inconsistent shares are detected via failed reconstruction (Algorithm 1, Line 10), using the Johnson-Lindenstrauss lemma for error detection without vector commitments. Honest dealers guarantee unique polynomials.

For secrecy, adversaries view $\leq t$ shares. MatrixHint-MLWE hides $\mathbf{x}$ via $\mathbf{u} = \mathbf{R}\mathbf{x} + \mathbf{y}$, with $\mathbf{y} \sim D_{\sigma_y}^d$ masking it. VE's IND-CPA secures decomposed shares, with negligible leakage. Detailed indistinguishability and soundness proofs are in Appendices B and C. $\square$

AV3S optimizes PV3S by deferring VE proofs to at most t shares, reducing computation.

Lemma 2. *AV3S satisfies correctness, completeness, secrecy, and termination under PV3S assumptions.*

Proof. Correctness and termination align with PV3S, using private decryption for signed shares and VE for supplementary ones. The Δ timer ensures async termination, defaulting to t proofs in the worst case.

Completeness is achieved via $n - t$ signatures and supplementary shares covering all indices ($S \cup P = [n]$).

Secrecy holds as deferred proofs expose $\leq t$ encrypted shares, below the $t + 1$ threshold, inheriting PV3S secrecy via VE's IND-CPA. $\square$

5.2. Analysis of AACS

AACS adapts Dumbo's ACS [25] with Delphi's approximate agreement [23] to ensure determinism.

Theorem 1. *The Approximate Asynchronous Common Subset (AACS) protocol satisfies approximate agreement, relaxed validity, and totality, with inconsistent outputs bounded by:*

$$P_{\text{AACS}} \leq P_{\text{wc}} + \kappa \cdot P_{\text{single}}, \quad (6)$$

where P_{wc} is the weak coin inconsistency probability ($\leq \alpha$), P_{single} is the per-BinAA instance inconsistency probability, and κ is the number of leaders.

Proof. As outlined in Section 3.3, BinAA uses inputs from two values: $\{0, 1\}$, with outputs o_i interpreted as 1 ($o_i \geq \text{MID}$) or 0 ($o_i < \text{MID}$), where $\text{MID} = 0.5$. If a nodes input 1 and $n - a$ input 0 (all honest), inconsistency occurs when the ϵ -bounded output range $[o_{\min}, o_{\max}]$ spans MID, with $P_{\text{inc}}(a)$ as the likelihood, constrained by BinAA's ϵ -agreement ($|o_i - o_j| < \epsilon$).

The output range centers around $\bar{o} = \frac{a}{n}$. Inconsistency requires $|\bar{o} - \text{MID}| \leq \epsilon/2$. When $a = n/2$, $\bar{o} = \text{MID}$, maximizing inconsistency if ϵ is large. As a deviates, $\bar{o}$ shifts, reducing inconsistency to zero. We obtain the inconsistency condition:

$$|a - n/2| \leq \frac{n\epsilon}{2}. \quad (7)$$

By setting ϵ small enough, we ensure $a = n/2$ (for even n) is the only integer satisfying this, making $P_{\text{inc}}(a) = 1$ at $a = n/2$ and 0 otherwise.

We compute $P(a = n/2)$ (assuming even n), influenced by Dumbo's early stopping strategy [25]: once any ABA outputs 1, nodes input 0 to remaining ABAs. For κ BinAA instances, each tied to a leader, preparation time $t_{i,k}$ for node i in instance k follows $t_{i,k} \sim \exp(1)$, modeling asynchronous delays.

Each BinAA terminates after constant c rounds [23], with termination time $T_j = \max_{i=1}^n t_{i,j} + c$. Early stopping occurs at $T_{\min} = \min_{j=1}^{\kappa} T_j$, after which unprepared nodes input 0. Each BinAA is first to terminate with probability $1/\kappa$, yielding $a = n$. Otherwise, with probability $(\kappa - 1)/\kappa$, $a \sim \text{Binomial}(n, p)$, where $p = 1 - e^{-T_{\min}}$.

Using extreme value theory, $\max_i t_{i,j} \approx \ln n + \gamma$ (Gumbel distribution [37], $\gamma \approx 0.577$), so:

$$E[T_{\min}] \approx \ln n - \ln \kappa + \gamma + c.$$

For instance, $n = 100$, $\kappa = 40$, $E[T_{\min}] \approx 1.493 + c$.

The total probability is:

$$P(a = n/2) = \frac{1}{\kappa} \cdot 0 + \frac{\kappa - 1}{\kappa} \cdot E \left[\binom{n}{n/2} p^{n/2} (1 - p)^{n/2} \right],$$

with expectation over $T_{\min}$ ($p = 1 - e^{-T_{\min}}$). For large n , we approximate $\text{Bin}(n, p) \approx \mathcal{N}(np, np(1 - p))$, giving:

$$P(a = n/2) \approx \frac{\kappa - 1}{\kappa} \cdot E \left[\frac{1}{\sqrt{2\pi np(1 - p)}} \exp \left(-\frac{(n/2 - np)^2}{2np(1 - p)} \right) \right]. \quad (8)$$

Let $\delta = (n/2 - np)/\sqrt{np(1 - p)}$. Since $E[p] \approx 1 - e^{-(1.493+c)} > 1/2$ for $c \geq 0$, $np > n/2$, so $\delta < 0$ and large, reducing the probability.

For $c = 0$, $E[T_{\min}] \approx 1.493$, $E[p] \approx 0.775$, $E[np] \approx 77.5$, $\delta \approx (50 - 77.5)/4.18 \approx -6.59$, and:

$$\exp \left(-\frac{(-6.59)^2}{2} \right) \approx \exp(-21.7) \approx 3.8 \times 10^{-10}.$$

With density ≈ 0.095 , $P(a = n/2) \approx 0.975 \times 3.6 \times 10^{-11} \approx 3.5 \times 10^{-11}$. For $c = 1$, $E[p] \approx 0.917$, $\delta \approx -15.1$, yielding $P(a = n/2) \approx 10^{-50}$. Larger c (e.g., $c = 5$) drives $P \rightarrow 0$ faster.

Early stopping biases a toward high values (near n) for non-first BinAAs, keeping $P(a = n/2) < 10^{-10}$. The overall AACS inconsistency probability P_{AACS} bounds the chance of any κ BinAA instances or the weak coin being inconsistent. With $P_{\text{single}} = P(a = n/2)$, and assuming independence, we use the union bound:

$$P_{\text{AACS}} \leq P_{\text{wc}} + 1 - (1 - P_{\text{single}})^{\kappa} < P_{\text{wc}} + \kappa \cdot P_{\text{single}}, \quad \square$$

By tuning parameters carefully, both P_{wc} and P_{single} can be sufficiently small to ensure P_{AACS} is negligible.

5.3. Security of LADKG

DKG provides the followings:

- **Correctness:** Honest nodes output consistent $(s_j, \mathbf{k})$, with $\mathbf{k} = \beta - [1 \ a] \cdot \mathbf{z}(0)$ for random degree- t $\mathbf{z}(x)$.
- **Completeness:** After $n - t$ sharings, a unique reconstructible $\mathbf{z}(x)$ exists.
- **Secrecy:** Adversary learns nothing about $\mathbf{z}(0)$ beyond $\mathbf{k}$, under t corruptions.
- **Termination:** Honest nodes terminate in async networks.

Theorem 2. *LADKG satisfies correctness, completeness, secrecy, and termination under Ring-LWE and MatrixHint-MLWE, with negligible failure probability in λ .*

Proof. AACS ensures a common $n - t$ transcript subset (totality, expected $O(\log n)$ rounds), with AV3S enabling share verification and Reed-Solomon reconstruction from $2t + 1$ shares.

Relaxed validity yields at least $n - 2t$ honest contributions. Selecting $t + 1$ transcripts guarantees one honest

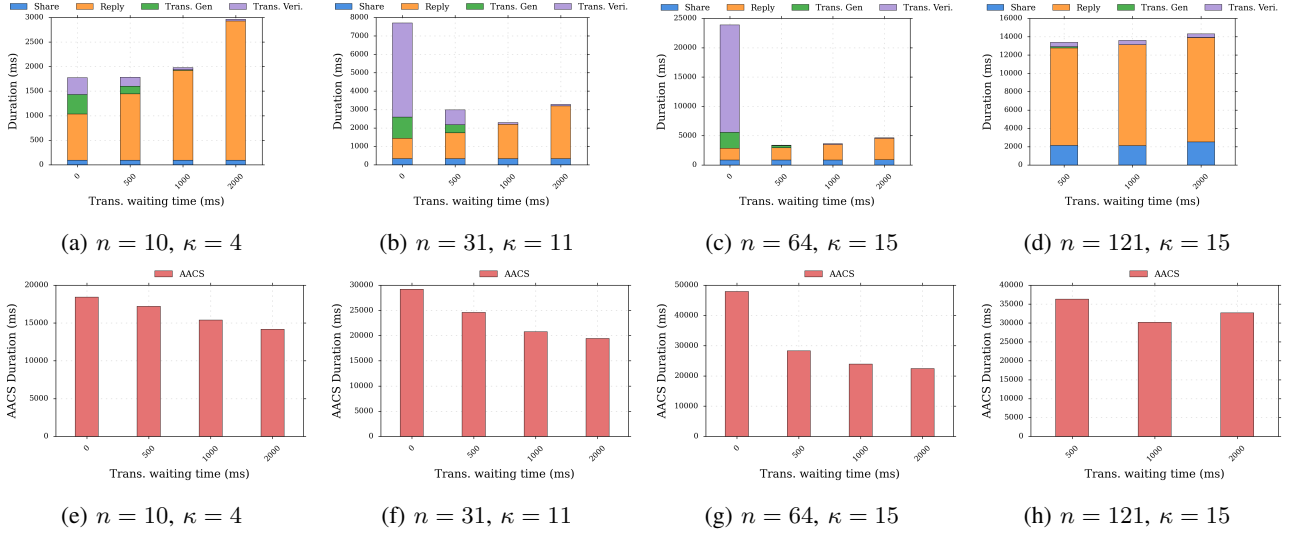


Figure 4: AV3S and AACS runtime breakdowns across different node counts: 10, 31, 64, 121 and κ . Top row: AV3S runtimes (colors: Share blue, Reply orange, Transcript Generation green, Transcript Verification violet). Bottom row: AACS runtime.

dealer; aggregation forms consistent $\mathbf{z}(x)$ shares. Reed-Solomon corrects t errors for unique $\mathbf{z}(0)$; weak coin salt a provides uniform blinding, with verifiable $\mathbf{k}$.

For secrecy, adversary sees $\leq t$ full secrets and encrypted shares. Simulator samples Gaussians for honest dealers, simulates AV3S/AACS views (via MatrixHint-MLWE/Ring-LWE), and adjusts extracted shares to match ideal $\mathbf{z}(0)$, indistinguishable due to masking. The detailed proof for secrecy is provided in Appendix B. $\square$

6. Evaluations

6.1. Experimental Environment Configuration

We evaluate our protocols in a geo-distributed setting using Amazon Web Services (AWS) EC2 instances of type t3.medium (each with 2 cores and 4 GB RAM), distributed across five regions: us-east-1, us-west-1, eu-north-1, ca-central-1, and eu-central-1. Each instance deploys a single node, running Ubuntu 20.04, with a burstable performance bandwidth limit of 5 Gbps. The nodes form a fully connected overlay network across these regions. Experiments are conducted with node counts $n = 10, 31, 64$, and 121. The implementation is in Rust, utilizing tokio as the runtime, SHA256 as the hash algorithm, and Dilithium as the post-quantum secure signature scheme. Forking from Delphi, our project is open-source to facilitate replication, with the code available at [github](https://github.com/anonymous.4open.science/r/ladkg-rs-6FB1/README.md)¹.

6.2. Parameter Selection

Ring Parameters: For the ring $\mathbb{Z}_q[x]/(x^n + 1)$, we select q as a prime near 2^{21} satisfying $q \equiv 5 \pmod{8}$ to support Verifiable Encryption (VE), with $n = 256$. This choice is constrained by the computational capabilities of the AWS instances, as discussed in the limitations. We employ standard convolution multiplication instead of

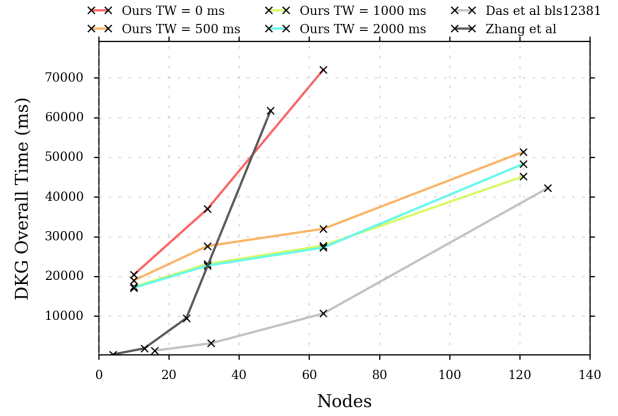


Figure 5: DKG runtime comparison across node counts, with TW variations for our scheme and baselines [4], [5].

NTT, as $q \equiv 5 \pmod{8}$ does not support it, and alternative methods introduce undue complexity.

BinAA Parameters: Following [23], each BinAA instance requires $\lceil \log_2(1/\epsilon) \rceil$ rounds. To minimize the total round complexity, we choose the largest ϵ for which the consistency condition (7) still holds—namely, $|a - n/2| < 1$.

AACS Parameters: For small n , we set $\kappa = t + 1$ to ensure a 100% probability of including at least one honest leader. For larger n , we select κ such that the probability of at least one honest leader, given by $P(\text{at least one honest}) = 1 - \frac{\binom{t}{\kappa}}{\binom{n}{\kappa}}$, approaches 1. For $n = 121$ with $t = 40$, setting $\kappa = 15$ yields $P(\text{at least one honest}) \approx 1 - 1.2 \times 10^{-6}$, ensuring high reliability.

AV3S Parameters: For the transcript waiting time (TW, i.e., Δ), we evaluate values of 0, 500, 1000, and 2000 ms, spanning optimistic to pessimistic scenarios. Although we tested a broader range, these values prove

1. <https://anonymous.4open.science/r/ladkg-rs-6FB1/README.md>

TABLE 2: Size of shares, supplementary shares, and secret/public key pairs.

Nodes	share (KB)	sup_share (KB)	secret key public key (KB)
10	364.305	1014.363	
31	420.774	1014.396	
64	509.494	1014.428	5.352 / 2.676
121	662.714	1014.460	

most informative: smaller TW approximates the worst-case behavior with t malicious nodes or high delays (equivalent to TW=0), while excessively large TW introduces unnecessary idle periods despite moderate latencies.

Norm Bound: To apply the large norm detection property from Lemma 2 in [10], we set the bound $b \leq q/(82n)$. This ensures that for a secret s with $\|s\| \geq b$, the probability that $\|\mathbf{R} \cdot s + \mathbf{y} \bmod q\| < \frac{1}{2}b\sqrt{26}$ is less than 2^{-128} ($B' = \frac{1}{2}b\sqrt{26}$). This allows us to detect large norms during verification while maintaining the required security margins for our parameter sets.

6.3. Experimental Results and Analysis

We analyze the performance of our AV3S and AACS protocols through runtime breakdowns and overall DKG efficiency. For AV3S, the runtime results (top row of Fig. 4) indicate that with smaller node counts ($n = 10$), varying Δ has minimal impact, as the number of supplementary shares in transcripts is low, and larger Δ values increase overall latency due to unnecessary waiting. However, as n increases ($n = 31, 64$), an optimal Δ significantly reduces Transcript Generation and Verification times, with $\Delta = 1000$ ms yielding the lowest latency, enabling near-instant transcript generation and balancing computational overhead with network delays.

For AACS, the runtime (bottom row of Fig. 4) is influenced by κ but predominantly by transcript size, as larger transcripts consume more Reliable Broadcast (RBC) bandwidth, slowing termination. For $n = 121$, the transcript size exceeds the bandwidth capacity of t3.medium instances, preventing completion of the $\Delta = 0$ case.

Fig. 5 reveals that our LADKG scheme exhibits rapid runtime growth under pessimistic conditions ($\Delta = 0$), exceeding 70 seconds at $n = 64$. However, under optimistic settings (e.g., $\Delta = 1000$ ms), it demonstrates better scalability compared to classical baseline ADKG schemes [4], [5]. Table 2 shows that the size of individual shares and supplementary shares increases with n , primarily due to the growing Merkle proof overhead, while the threshold key pair size remains fixed.

6.4. Limitations

The Verifiable Encryption scheme requires a modulus $q \equiv 5 \pmod{8}$, which precludes direct use of the Number Theoretic Transform (NTT) for efficient polynomial multiplication, as NTT demands $q \equiv 1 \pmod{2n}$ for full splitting of $x^n + 1$ into linear factors. Consequently, the ring is decomposed via the Chinese Remainder Theorem (CRT) into two subrings of dimension $n/2$, with multiplications performed independently using optimized techniques for NTT-unfriendly settings, such as incomplete

NTT combined with Karatsuba [38] or schoolbook methods. This increases computational overhead compared to NTT-friendly parameters.

Due to the computational constraints of AWS EC2 t3.medium instances, smaller ring parameters ($q \approx 2^{21}$, $n = 256$) were selected, satisfying $q \equiv 5 \pmod{8}$ for VE support. While sufficient for proof-of-concept evaluations, these yield reduced security levels; production deployments should employ larger values (e.g., $q \approx 2^{50}$, $n = 1024$) to achieve adequate post-quantum security margins.

In our AV3S implementation, the certificate is formed by directly collecting raw signatures rather than aggregating them, leading to larger certificate sizes. While post-quantum secure non-sequential aggregate signature schemes, such as those proposed in [39]–[41], could address this, we do not adopt lattice-based aggregate signatures due to the preliminary state of the Rust ecosystem, which complicates their implementation. We provide the primitive for aggregatable signatures in Appendix A.

The scalability of LADKG is constrained by the size of transcripts in the AACS phase, which grows linearly with the number of nodes due to the inclusion of supplementary shares. Each sup_share approaches 1 MB in size, and in the pessimistic case, a transcript may contain up to $n/3$ such shares, leading to substantial bandwidth demands during Reliable Broadcast (RBC). While optimistic settings ($\Delta > 0$) mitigate this by reducing the number of sup_share through additional signatures, the overall approach incurs high communication costs in large-scale systems.

7. Related Works

Distributed Key Generation protocols have evolved significantly over the past decades, transitioning from foundational synchronous constructions in pre-quantum cryptography to more resilient asynchronous variants, and recently extending into post-quantum paradigms based on lattice assumptions.

DLog-Based Distributed Key Generation: Early DKG protocols targeted synchronous networks, leveraging bounded delays for fault detection and termination. Pedersen’s 1992 protocol [42] enabled n parties to generate a shared discrete logarithm-based key pair without a trusted dealer, establishing threshold cryptography foundations but with limited fault tolerance and a trusted setup. Later works integrated Publicly Verifiable Secret Sharing (PVSS) for enhanced verifiability. Gurkan et al. [43] achieved $O(\log n)$ malicious party tolerance, while Groth’s non-interactive DKG [44] eliminated post-setup rounds for static networks.

Recent synchronous protocols prioritize communication efficiency. Shrestha et al. [45] proposed a non-broadcast DKG avoiding Byzantine broadcasts, with $O(\lambda n^3)$ complexity—quadratic over prior $O(n^4)$ schemes—and Identifiable Abort (IA) support.

Asynchronous DKG (ADKG) emerged to handle arbitrary network delays. Kokoris-Kogias et al.’s initial design [46] incurred $O(\lambda n^4)$ communication and $O(n)$ expected rounds. Abraham et al. [2] reduced this to $O(\lambda n^3 \log n)$, Das et al. [47] to $O(\lambda n^3)$, and a recent near-quadratic variant [3] to $\tilde{O}(n^{2+\epsilon})$.

These protocols now support a wide range of network models and fault tolerances, with significant reductions in communication and computational overhead. However, achieving similar desirable properties in lattice-based DKG is challenging due to inherent constraints, such as noise amplification issues, and non-negligible computational costs.

Lattice-Based Distributed Key Generation: Lattice-based cryptography promises post-quantum DKG, yet the field is nascent. Bendlin et al. [48] (2009) pioneered lattice threshold decryption, noting DKG feasibility without specifics. Alborch et al. [49] delivered the first R-LWE-based LDKG for basic threshold keys. Geihs et al. [50] introduced an LWE-based distributed PRF for efficient LDKG and constant-storage key derivation in 8 online rounds. Ji et al. [9] provided the first robust t -out-of- n lattice threshold signature with LDKG, using Fiat-Shamir with Aborts and complaint-based VSS for $t < n/3$ tolerance in synchronous settings. Espitau et al. [10] offered a concurrent robust hash-and-sign scheme with integrated LDKG, featuring Verifiable Short Secret Sharing (V3S) to curb Shamir-induced norm growth. However, these lattice-based DKG protocols generally do not perform well in asynchronous networks, limiting their applicability in real-world distributed systems.

In parallel, lattice-based threshold signatures have seen notable advancements. Pino et al.'s three-round Identifiable Abort (IA) protocol TRaccoon [7] optimized noise for low latency; Gur et al. [27] enabled two-round signing with content-independent first messages. Pino et al.'s latest compact scheme [26] yields Dilithium-sized signatures (up to 8 parties) via a two-round LDKG aligned with NIST standards. These threshold signature schemes often treat key generation as a primitive, assuming a pre-distributed threshold key pair. This underscores the importance of dedicated lattice-based DKG protocols, which provide secure and efficient key setup without relying on trusted dealers.

8. Conclusion

In this paper, we introduce the Lattice-Based Asynchronous Distributed Key Generation (LADKG) protocol designed to enhance post-quantum distributed key generation in asynchronous network settings. By integrating Asynchronous Verifiable Short Secret Sharing and an Approximate Asynchronous Common Subset protocol, LADKG ensures correctness, completeness, secrecy, and termination under the Ring-LWE and MatrixHint-MLWE hardness assumptions. Experimental evaluations indicate that LADKG achieves scalability and efficiency comparable to established classical ADKG protocols.

Acknowledgment

This work is supported in part by National Key R&D Program of China, under Grant 2023YFB2703800, in part by the Tianjin Natural Science Foundation 23JCY-BJC01750, in part by the National Science Foundation of China under Grants U23B2049, U22B2027, U2468204 and 62172297, the China Guangxi Science and Technology Plan Project (Guangxi Science and

Technology Base and Talent Special Project) under Grant AD23026096 (Application Number 2022AC20001), hainan Province Science and Technology Special Fund (Grant No. ZDYF2024GXJS008), the Tianjin Special Fund Project for High-quality Development of Manufacturing Industry under Grants 20241048 and 20251148, Tianjin Science and Technology Plan Project under Grant 23YDPYGX00140.

References

- [1] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, "Secure distributed key generation for discrete-log based cryptosystems," in *Advances in Cryptology — EUROCRYPT '99*, J. Stern, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 295–310.
- [2] I. Abraham, P. Jovanovic, M. Maller, S. Meiklejohn, G. Stern, and A. Tomescu, "Reaching consensus for asynchronous distributed key generation," in *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, ser. PODC'21. New York, NY, USA: Association for Computing Machinery, 2021, p. 363–373. [Online]. Available: <https://doi.org/10.1145/3465084.3467914>
- [3] I. Abraham, R. Bacho, J. Loss, and G. Stern, "Nearly quadratic asynchronous distributed key generation," *Cryptology ePrint Archive*, Paper 2025/006, 2025. [Online]. Available: <https://eprint.iacr.org/2025/006>
- [4] S. Das, Z. Xiang, L. Kokoris-Kogias, and L. Ren, "Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 5359–5376. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/das>
- [5] H. Zhang, S. Duan, C. Liu, B. Zhao, X. Meng, S. Liu, Y. Yu, F. Zhang, and L. Zhu, "Practical asynchronous distributed key generation: Improved efficiency, weaker assumption, and standard model," in *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2023, pp. 568–581.
- [6] L. Chen, L. Chen, S. Jordan, Y.-K. Liu, D. Moody, R. Peralta, R. A. Perlner, and D. Smith-Tone, *Report on post-quantum cryptography*. US Department of Commerce, National Institute of Standards and Technology ..., 2016, vol. 12.
- [7] R. del Pino, S. Katsumata, G. Niot, M. Reichle, and K. Takemure, "Unmasking TRaccoon: A lattice-based threshold signature with an efficient identifiable abort protocol," *Cryptology ePrint Archive*, Paper 2025/849, 2025. [Online]. Available: <https://eprint.iacr.org/2025/849>
- [8] R. del Pino, T. Espitau, G. Niot, and T. Prest, "Simple and efficient lattice threshold signatures with identifiable aborts," *Cryptology ePrint Archive*, Paper 2025/871, 2025. [Online]. Available: <https://eprint.iacr.org/2025/871>
- [9] Y. Ji, Y. Tao, and R. Zhang, "Robust (t, n) -threshold lattice signature," *Available at SSRN* 4588269.
- [10] T. Espitau, G. Niot, and T. Prest, "Flood and submerge: Distributed key generation and robust threshold signature from lattices," in *Advances in Cryptology — CRYPTO 2024*, L. Reyzin and D. Stebila, Eds. Cham: Springer Nature Switzerland, 2024, pp. 425–458.
- [11] X. Wang, S. Duan, J. Clavin, and H. Zhang, "Bft in blockchains: From protocols to use cases," *ACM Comput. Surv.*, vol. 54, no. 10s, Sep. 2022. [Online]. Available: <https://doi.org/10.1145/3503042>
- [12] S. Gupta, J. Hellings, S. Rahnama, and M. Sadoghi, "An in-depth look of bft consensus in blockchain: Challenges and opportunities," in *Proceedings of the 20th International Middleware Conference Tutorials*, ser. Middleware '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 6–10. [Online]. Available: <https://doi.org/10.1145/3366625.3369437>
- [13] G. Zhang, F. Pan, Y. Mao, S. Tjanic, M. Dang'ana, S. Motepalli, S. Zhang, and H.-A. Jacobsen, "Reaching consensus in the byzantine empire: A comprehensive review of bft consensus algorithms," *ACM Comput. Surv.*, vol. 56, no. 5, Jan. 2024. [Online]. Available: <https://doi.org/10.1145/3636553>

- [14] F. Gai, J. Niu, I. Beschastnikh, C. Feng, and S. Wang, "Scaling blockchain consensus via a robust shared mempool," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 530–543.
- [15] S. Das, Z. Xiang, A. Tomescu, A. Spiegelman, B. Pinkas, and L. Ren, "Verifiable secret sharing simplified," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 633–651.
- [16] B. Abdolmaleki, J. Clark, M. Foroutani, S. Khazaei, and S. Nasirzadeh, "A generic framework for practical lattice-based non-interactive publicly verifiable secret sharing," *Cryptology ePrint Archive*, Paper 2025/901, 2025. [Online]. Available: <https://eprint.iacr.org/2025/901>
- [17] P. N. Minh, K. Nguyen, W. Susilo, and K. Nguyen-An, "Publicly verifiable secret sharing: Generic constructions and lattice-based instantiations in the standard model," 2025. [Online]. Available: <https://arxiv.org/abs/2504.14381>
- [18] S. Das, T. Yurek, Z. Xiang, A. Miller, L. Kokoris-Kogias, and L. Ren, "Practical asynchronous distributed key generation," in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 2518–2534.
- [19] Y. Gao, Y. Lu, Z. Lu, Q. Tang, Y. Wang, and J. Xu, "Turritopsis: Practical dynamic asynchronous bft," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 5588–5603, 2025.
- [20] B. Guo, Y. Lu, Z. Lu, Q. Tang, J. Xu, and Z. Zhang, "Speeding dumbos: Pushing asynchronous BFT closer to practice," *Cryptology ePrint Archive*, Paper 2022/027, 2022. [Online]. Available: <https://eprint.iacr.org/2022/027>
- [21] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *J. ACM*, vol. 32, no. 2, p. 374–382, Apr. 1985. [Online]. Available: <https://doi.org/10.1145/3149.214121>
- [22] I. Abraham, Y. Amit, and D. Dolev, "Optimal resilience asynchronous approximate agreement," in *Principles of Distributed Systems*, T. Higashino, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 229–239.
- [23] A. Bandrupalli, A. Bhat, S. Bagchi, A. Kate, C.-D. Liu-Zhang, and M. K. Reiter, "Delphi: Efficient asynchronous approximate agreement for distributed oracles," in *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2024, pp. 456–469.
- [24] V. Lyubashevsky and G. Neven, "One-shot verifiable encryption from lattices," in *Advances in Cryptology – EUROCRYPT 2017*, J.-S. Coron and J. B. Nielsen, Eds. Cham: Springer International Publishing, 2017, pp. 293–323.
- [25] B. Guo, Z. Lu, Q. Tang, J. Xu, and Z. Zhang, "Dumbos: Faster asynchronous bft protocols," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 803–818. [Online]. Available: <https://doi.org/10.1145/3372297.3417262>
- [26] R. del Pino and G. Niot, "Finally! a compact lattice-based threshold signature," in *Public-Key Cryptography – PKC 2025: 28th IACR International Conference on Practice and Theory of Public-Key Cryptography, Røros, Norway, May 12–15, 2025, Proceedings, Part III*. Berlin, Heidelberg: Springer-Verlag, 2025, p. 169–199. [Online]. Available: https://doi.org/10.1007/978-3-031-91826-1_6
- [27] K. D. Gur, J. Katz, and T. Silde, "Two-round threshold lattice-based signatures from threshold homomorphic encryption," in *Post-Quantum Cryptography*, M.-J. Saarinen and D. Smith-Tone, Eds. Cham: Springer Nature Switzerland, 2024, pp. 266–300.
- [28] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, "Secure and efficient asynchronous broadcast protocols," in *Advances in Cryptology – CRYPTO 2001*, J. Kilian, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 524–541.
- [29] C. Cachin, K. Kursawe, and V. Shoup, "Random oracles in constantipole: practical asynchronous byzantine agreement using cryptography (extended abstract)," in *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing*, ser. PODC '00. New York, NY, USA: Association for Computing Machinery, 2000, p. 123–132. [Online]. Available: <https://doi.org/10.1145/343477.343531>
- [30] I. Abraham, D. Malkhi, and A. Spiegelman, "Asymptotically optimal validated asynchronous byzantine agreement," in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, ser. PODC '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 337–346. [Online]. Available: <https://doi.org/10.1145/3293611.3331612>
- [31] G. Bracha, "Asynchronous byzantine agreement protocols," *Information and Computation*, vol. 75, no. 2, pp. 130–143, 1987. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/089054018790054X>
- [32] R. J. McEliece and D. V. Sarwate, "On sharing secrets and reed-solomon codes," *Commun. ACM*, vol. 24, no. 9, p. 583–584, Sep. 1981. [Online]. Available: <https://doi.org/10.1145/358746.358762>
- [33] W. Johnson, J. Lindenstrauss, and G. Schechtman, "Extensions of lipschitz maps into banach spaces," *Israel Journal of Mathematics*, vol. 54, pp. 129–138, 06 1986.
- [34] A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song, "The honey badger of bft protocols," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 31–42. [Online]. Available: <https://doi.org/10.1145/2976749.2978399>
- [35] A. Bandrupalli, A. Bhat, S. Bagchi, A. Kate, and M. K. Reiter, "Random beacons in monte carlo: Efficient asynchronous random beacon without threshold cryptography," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2621–2635. [Online]. Available: <https://doi.org/10.1145/3658644.3670326>
- [36] M. F. Esgin, T. Espitau, G. Niot, T. Prest, A. Sakzad, and R. Steinfeld, "Plover: Masking-friendly hash-and-sign lattice signatures," in *Advances in Cryptology – EUROCRYPT 2024*, M. Joye and G. Leander, Eds. Cham: Springer Nature Switzerland, 2024, pp. 316–345.
- [37] E. J. Gumbel, "The return period of flood flows," *The Annals of Mathematical Statistics*, vol. 12, no. 2, pp. 163–190, 1941. [Online]. Available: <http://www.jstor.org/stable/2235766>
- [38] A. Karatsuba, "Multiplication of multidigit numbers on automata," in *Soviet physics doklady*, vol. 7, 1963, pp. 595–596.
- [39] M. A. Aardal, D. F. Aranha, K. Boudgoust, S. Kolby, and A. Takahashi, "Aggregating falcon signatures with LaBRADOR," *Cryptology ePrint Archive*, Paper 2024/311, 2024. [Online]. Available: <https://eprint.iacr.org/2024/311>
- [40] T. Tomita and J. Shikata, "Compact aggregate signature from module-lattices," *Cryptology ePrint Archive*, Paper 2023/471, 2023. [Online]. Available: <https://eprint.iacr.org/2023/471>
- [41] J. Drake, D. Khovratovich, M. Kudinov, and B. Wagner, "Hash-based multi-signatures for post-quantum ethereum," *Cryptology ePrint Archive*, Paper 2025/055, 2025. [Online]. Available: <https://eprint.iacr.org/2025/055>
- [42] T. P. Pedersen, "A threshold cryptosystem without a trusted party," in *Advances in Cryptology – EUROCRYPT '91*, D. W. Davies, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1991, pp. 522–526.
- [43] K. Gurkan, P. Jovanovic, M. Maller, S. Meiklejohn, G. Stern, and A. Tomescu, "Aggregatable distributed key generation," in *Advances in Cryptology – EUROCRYPT 2021*, A. Canteaut and F.-X. Standaert, Eds. Cham: Springer International Publishing, 2021, pp. 147–176.
- [44] J. Groth, "Non-interactive distributed key generation and key resharing," *Cryptology ePrint Archive*, Paper 2021/339, 2021. [Online]. Available: <https://eprint.iacr.org/2021/339>
- [45] N. Shrestha, A. Bhat, A. Kate, and K. Nayak, "Synchronous distributed key generation without broadcasts," *Cryptology ePrint Archive*, Paper 2021/1635, 2021. [Online]. Available: <https://eprint.iacr.org/2021/1635>
- [46] E. Kokoris Kogias, D. Malkhi, and A. Spiegelman, "Asynchronous distributed key generation for computationally-secure randomness, consensus, and threshold signatures," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1751–1767. [Online]. Available: <https://doi.org/10.1145/3372297.3423364>

- [47] S. Das, Z. Xiang, and L. Ren, “Asynchronous data dissemination and its applications,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 2705–2721. [Online]. Available: <https://doi.org/10.1145/3460120.3484808>
- [48] R. Bendlin and I. Damgård, “Threshold decryption and zero-knowledge proofs for lattice-based cryptosystems,” vol. 2009, 01 2009, p. 391.
- [49] F. Alborch, R. Martínez, and P. Morillo, “R-Iwe-based distributed key generation and threshold decryption,” *Mathematics*, vol. 10, no. 5, 2022. [Online]. Available: <https://www.mdpi.com/2227-7390/10/5/728>
- [50] M. Geihs and H. Montgomery, “LaKey: Efficient Lattice-Based distributed PRFs enable scalable distributed key management,” in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 4319–4335. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/geihs>

Appendix

1. Aggregatable Signature Scheme

An aggregatable signature scheme (AS) consists of the following tuple of algorithms:

- $\text{pp} \leftarrow \text{Setup}(\lambda)$: A setup algorithm that produces public protocol parameters pp based on a security parameter λ .
- $((\text{pk}, \pi), \text{sk}) \leftarrow \text{GenKeyPair}(\text{pp})$: A key pair generation algorithm that yields a secret key sk alongside a corresponding public key pk and a proof π of secret key possession.
- $0 \text{ or } 1 \leftarrow \text{VerifySK}(\text{pp}, \text{pk}, \pi)$: A verification algorithm that, given a public key pk and proof-of-possession π , returns 1 if π validates pk , and 0 otherwise.
- $\sigma \leftarrow \text{Sign}(\text{pp}, \text{sk}, m)$: A signing algorithm that, given a secret key sk and message $m \in \{0, 1\}^*$, generates a signature σ .
- $\text{apk} \leftarrow \text{AggrPK}(\text{pp}, (\text{pk}_i)_{i=1}^n)$: A public key aggregation algorithm that, given a set of public keys $(\text{pk}_i)_{i=1}^n$, computes an aggregate public key apk .
- $\text{asig} \leftarrow \text{AggrSig}(\text{pp}, (\sigma_i)_{i=1}^n)$: A signature aggregation algorithm that, given a collection of signatures $(\sigma_i)_{i=1}^n$, produces an aggregate signature asig .
- $0 \text{ or } 1 \leftarrow \text{Verify}(\text{pp}, \text{apk}, m, \text{asig})$: A signature verification algorithm that, given an aggregate public key apk , message $m \in \{0, 1\}^*$, and aggregate signature asig , returns 1 if valid, and 0 otherwise.

In our AV3S protocol, this AS scheme can be integrated as follows: the process begins with AS.Setup and AS.VerifySK to establish the scheme’s foundation. In AV3S.Verify , the signature format remains unchanged. However, in AV3S.Confirm , the dealer aggregates collected signatures using AS.AggrSig upon receiving at least $2t + 1$ signatures, producing an aggregate signature asig along with a list of corresponding node IDs, forming the certificate cert instead of a raw signature set. In $\text{AV3S.TranscriptVerify}$, nodes aggregate the public keys corresponding to the node IDs using AS.AggrPK and verify the cert ’s validity using AS.Verify .

2. Proofs for Indistinguishability

In this appendix, we provide security proofs for the indistinguishability properties of PV3S and AV3S, which are crucial for the secrecy of LADKG. We formalize the security through two indistinguishability games and show that any adversary breaking the secrecy must win at least one of these games with non-negligible probability.

Theorem 3. *If the secrecy of LADKG does not hold—i.e., there exists a PPT adversary $\mathcal{A}$ that learns non-negligible information about the aggregated secret $\mathbf{z}(0)$ beyond the public key $\mathbf{k}$ under at most t corruptions—then $\mathcal{A}$ must win at least one of the following games with non-negligible probability:*

- 1) *Indistinguishability of shares (Game 1).*
- 2) *Indistinguishability of supplementary shares (Game 2).*

We first define the simulators used in the games.

Algorithm 4 SimShare(S): Simulator for Ordinary Shares

```

1: for  $i \in S$  do
2:   Sample  $\mathbf{m}_i \sim S_\gamma^{k+\ell}$  ▷ Real witnesses
3: end for
4: for  $i \in [N] \setminus S$  do
5:   Sample  $\mathbf{m}_i \sim R_q^{k+\ell}$  ▷ Fake witnesses
6: end for
7: for  $i \in [N]$  do
8:    $(\text{ct}_i, \text{st}_i) \leftarrow \text{RLWE-Enc}(pk_i, \mathbf{m}_i)$ 
9: end for
10: Build Merkle tree over all leaves (real for  $S$ , fake for others);
    obtain root  $h$  and proofs  $\{\text{mpf}_i\}_{i=1}^N$ 
11: Derive  $\mathbf{R} \leftarrow H_R(h)$ 
12: Build  $\mathbf{B} \leftarrow [\mathbf{R}\mathbf{G}_x \mid \mathbf{G}_y]$ 
13: for  $i \in S$  do
14:   Compute  $\mathbf{u}_i = \mathbf{B}\mathbf{m}_i \pmod{q}$ 
15: end for
16: for  $i \in [N] \setminus S$  do
17:   Sample  $\mathbf{u}_i \sim U(R_q^l)$ 
18: end for
19: return  $\{\text{ct}_i, \mathbf{u}_i, \text{mpf}_i, h\}_{i \in [N]}$ 

```

Algorithm 5 SimSecret($\{\text{ct}_i, \mathbf{u}_i, \text{mpf}_i, h\}_{i \in S}$): Simulator for Fake Secret

```

1:  $\mathbf{R} \leftarrow H_R(h)$ 
2:  $\mathbf{z} = \mathbf{u}_0$ 
3:  $\Sigma_0 = (\frac{1}{\sigma_x^2} \cdot \mathbf{I} + \frac{1}{\sigma_y^2} \cdot \mathbf{R}^T \mathbf{R})^{-1}$ 
4:  $\mathbf{c} = \Sigma_0 \cdot \frac{1}{\sigma_y^2} \cdot \mathbf{R}^T \cdot \mathbf{z}$ 
5: return  $\mathbf{x} \sim D_{R, \sqrt{\Sigma_0}, \mathbf{c}}$ 

```

This is identical to the SimSecret in [10], generating a fake short secret consistent with the observed encrypted share.

Algorithm 6 SimSupShare($\text{share}_i, \text{st}_i$): Simulator for Supplementary Shares

Input: VE public key pk_i

```

1: Derive  $\mathbf{R} \leftarrow H_R(h)$ ; build  $\mathbf{B} \leftarrow [\mathbf{R}\mathbf{G}_x \mid \mathbf{G}_y]$ 
2:  $\pi_i \leftarrow \text{VE-SimProve}(pk_i, \mathbf{B}, \mathbf{u}_i, \text{st}_i)$  ▷ Simulatable as in [24]
3: return  $\{\text{ct}_i, \mathbf{u}_i, \pi_i, \text{mpf}_i, h\}_{i \in P}$ 

```

We now define the two games.

2.1. Game 1: Indistinguishability of Ordinary Shares.

Definition 9 (Game 1). *The game proceeds as follows:*

- *Input:* $b \in \{0, 1\}$.
- *The adversary $\mathcal{A}$ chooses a set $S \subseteq [N]$, $|S| \leq t$, as the corrupted parties.*
- *If $b = 0$: Sample real secret $\mathbf{x} \sim D_{\sigma}^d$; generate real shares $\{\text{share}_i\}_{i \in [N]} \leftarrow \text{AV3S.Share}(N, t+1, \mathbf{x})$.*
- *If $b = 1$: Generate simulated shares $\{\text{share}_i\}_{i \in [N]} \leftarrow \text{SimShare}(S)$.*
- *$\mathbf{x} \leftarrow \text{SimSecret}(h, \{\mathbf{u}_i\}_{i=1}^N)$.*
- *Send $\mathbf{x}$ and the shares for $i \in S$ to $\mathcal{A}$.*
- *$\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$.*
- *$\mathcal{A}$ wins if $b' = b$.*

Lemma 3. *Under the Ring-LWE and MatrixHint-MLWE assumptions, no PPT adversary $\mathcal{A}$ can win Game 1 with probability greater than $1/2 + \text{negl}(\lambda)$.*

Proof. The proof follows a hybrid argument similar to the fragmentary knowledge proof in [10].

Hybrid 0: The real distribution ($b = 0$): Real shares from AV3S.Share .

Hybrid 1: Replace the Merkle leaves for $i \notin S$ with random hashes, as in SimShare . The Merkle tree is indistinguishable due to the collision resistance of the hash function.

Hybrid 2: Replace $\mathbf{u}_i$ for $i \notin S$ with uniform random in R_q^l . This is indistinguishable under MatrixHint-MLWE, as the submersion $\mathbf{R}$ and Gaussian blinding $\mathbf{y}$ hide the structure.

Hybrid 3: Replace the real secret with $\mathbf{x}$ from SimSecret . By the statistical hiding of discrete Gaussians, the distributions are statistically close.

Hybrid 3 matches the simulated distribution ($b = 1$). The total advantage is $\text{negl}(\lambda)$. $\square$

2.2. Game 2: Indistinguishability of Supplementary Shares.

Definition 10 (Game 2). *Starting from the simulated world of Game 1, The game proceeds as follows:*

- *Input:* $b \in \{0, 1\}$.
- *The adversary $\mathcal{A}$ chooses a set $S \subseteq [N]$, $|S| \leq t$, as the corrupted parties.*
- *Generate simulated ordinary shares for all parties: $\{\text{ct}_i, \mathbf{u}_i, \text{mpf}_i, h\}_{i \in [N]} \leftarrow \text{SimShare}(S)$. The adversary $\mathcal{A}$ receives the corresponding shares in S .*
- *$\mathcal{A}$ chooses $C \subseteq S$ (simulating the signatures returned to the dealer).*
- *Let $P = S \setminus C$. For $i \in P$:*
 - *Retrieve $\text{share}_i = \{\text{ct}_i, \mathbf{u}_i, \text{mpf}_i, h\}$ and st_i*
 - *If $b = 0$: Generate real supplementary shares sup_share_i using share_i as in AV3S.Confirm .*
 - *If $b = 1$: Generate simulated supplementary shares $\text{sup_share}_i \leftarrow \text{SimSupShare}(\text{share}_i, \text{st}_i)$.*
- *Send the supplementary shares for $i \in P$ to $\mathcal{A}$.*
- *$\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$ based on its full view.*

- *$\mathcal{A}$ wins if $b' = b$.*

Lemma 4. *Under the simulation-sound extractability of the VE proof system and the Ring-LWE assumption, no PPT adversary can win Game 2 with probability greater than $1/2 + \text{negl}(\lambda)$.*

Proof. We proceed with a hybrid argument within Game 2, starting from the simulated ordinary shares of Game 1.

Hybrid 0 (within Game 2): $b = 0$. Real VE proofs using true witnesses $\mathbf{m}_i$.

Hybrid 1: Replace the VE proofs π_i for $i \in P$ with simulated proofs using the zero-knowledge simulator of [24] and st_i . Indistinguishable under Ring-LWE (as the public key and statement are unchanged).

Hybrid 1 matches $b = 1$. Total advantage $\text{negl}(\lambda)$. $\square$

Proof of Theorem 3. In LADKG, the adversary's view consists of at most t ordinary shares and up to t supplementary shares from honest dealers. If secrecy is broken, $\mathcal{A}$ distinguishes real aggregated secrets from simulated ones. By reduction, this implies distinguishing in Game 1 (for ordinary shares) or Game 2 (for supplementary shares), contradicting Lemmas 3 and 4. $\square$

3. Proofs for AV3S Soundness

The correctness of LADKG relies on the soundness of AV3S, which ensures that a malicious dealer cannot produce inconsistent shares that pass verification. Soundness guarantees that if shares pass AV3S.Verify for a set S of at least $T = t + 1$ parties, then the reconstructed secret $\mathbf{x}$ is consistent, unique, and satisfies the short-norm bound (i.e., $\|\mathbf{x}\| \leq B$ for some bound B). We adapt the soundness notion and proof from the V3S scheme in [10] to our AV3S, incorporating the verifiable encryption (VE) checks.

Definition 11 (AV3S Soundness). *A malicious PPT adversary $\mathcal{A}$ (acting as dealer) has negligible advantage in producing N shares such that for some honest set $S \subseteq [N]$ with $|S| \geq T$:*

AV3S.Verify accepts for all $i \in S$, But either AV3S.Reconstruct on $\llbracket \mathbf{s} \rrbracket_{i \in S}$ returns $\perp$ (inconsistency), or the reconstructed $\mathbf{x}$ violates $\|\mathbf{x}\| \leq B$.

We formalize this via the following game in the Random Oracle Model (ROM).

Definition 12 (AV3S-Sound Game). *The game proceeds as follows:*

- *$\mathcal{A}$ chooses a set $S \subseteq [N]$, $|S| \geq T$.*
- *$\mathcal{A}^{H(\cdot)}(N, T, S)$ generates N share, where $\mathcal{A}^H$ is a PPT algorithm with bounded queries to the random oracle H .*
- *For each $i \in S$: If $\text{AV3S.Verify}(\text{share}_i)$ returns $\perp$, $\mathcal{A}$ loses.*
- *Compute $\mathbf{x} \leftarrow \text{AV3S.Reconstruct}(\llbracket \mathbf{s} \rrbracket_{i \in S})$.*
- *If $\mathbf{x} = \perp$ or $\|\mathbf{x}\| > B$, $\mathcal{A}$ wins; else $\mathcal{A}$ loses.*

A similar game exists for the soundness of transcripts generated in AV3S.Confirm , where $\mathcal{A}$ produces supplementary shares (including VE proofs π_i) for a subset $P \subseteq [N]$, $|P| \leq t$, and the game checks $\text{AV3S.TranscriptVerify}$ acceptance but reconstruction failure or norm violation.

The proof for this transcript-sound game follows analogously, leveraging the soundness of VE in addition to the base mechanisms.

Lemma 5. *Under the MatrixHint-MLWE assumption in the ROM, no PPT adversary $\mathcal{A}$ wins the AV3S-Sound Game with more than negligible advantage $\text{Adv}_{\mathcal{A}}(\lambda) \leq \text{negl}(\lambda)$.*

Proof. Let Q_H be the number of queries to the random oracle H .

Hybrid 0: The real AV3S-Sound Game.

Hybrid 1: Program the random oracle such that the submersion matrix $\mathbf{R} = H_R(h)$ is sampled after the shares are chosen. If $\mathcal{A}$ queries H on a colliding or preimage-violating input, $\mathcal{A}$ loses. The advantage loss is bounded by $Q_H^2/2^{2\lambda-1}$ due to hash collision/preimage resistance [10].

Hybrid 2: Modify the game to include assertions based on the distribution $\mathcal{D}_R$ properties (separation and large-norm detection). If the shares are inconsistent (failing separation) or the reconstructed $\mathbf{x}$ has large norm (failing detection), but verification passed, $\mathcal{A}$ wins early. Otherwise, the game aborts if assertions fail. By the properties of $\mathcal{D}_R$ (with loss factors p_1 for separation probability and p_2 for detection probability), the advantage is bounded by $Q_H \cdot (p_1 + p_2)$.

Hybrid 3: Replace the VE decryption checks in AV3S.Verify with simulated ones under Ring-LWE. The submersion and Gaussian blinding ensure that invalid (long-norm) decompositions are detected with overwhelming probability, adding negligible loss.

In Hybrid 3, if verification passes, the shares must be consistent and $\mathbf{x}$ short, so $\mathcal{A}$ cannot win except with negligible probability (union bound over queries and $\mathcal{D}_R$ losses). $\square$

4. Complexity Analysis

The communication in LADKG comprises four main phases: Share, Reply, AACS, and PKReconstruct. Each phase has a per-node message complexity of $O(n)$, as nodes exchange data pairwise or broadcast equivalents in the asynchronous setting. The largest messages occur in the AACS phase, where transcripts are disseminated. In the worst case, a transcript size is $O(\lambda n \log n)$, dominated by up to t supplementary shares (each containing VE proofs and ciphertexts of size $O(\lambda \log n)$). Thus, the per-node communication complexity is $O(\lambda n^2 \log n)$.

The computational complexity is primarily dominated by Merkle proof verification in AV3S.TranscriptVerify, which incurs $O(n \log n)$ per transcript. However, if RS error correction is required, the complexity increases, potentially reaching $O(n^3)$ when employing the Berlekamp-Welch algorithm. In practice, the dominant computational cost stems from VE-Prove operations, involving FSwA rejection sampling and high-dimensional matrix multiplications.